队伍编号	MCB2301622
赛道	B

基于指数平滑与 DBSCAN 的多重特征电商需求时序预测模型

摘 要

随着消费水平的提高，电商平台依托线上销售渠道的兴起，逐渐进入人们的视野，并且凭借它的数字化管理系统仓库，得到众多商家的青睐。因此及时存储处于空档期间的仓库、预知各地仓库的需求量对商家和客户来说，是双赢的局面。

针对问题一：解决各商家在各仓库的商品的需求量并根据商家-仓库-商品三个维度进行时序分类。首先进行数据预处理，包括：**剔除重复值**、**标签编码**、**异常值处理**等，利用**小提琴图**识别异常值确保数据的合理性，之后将数据转化为数值型方便后期处理。接着观察附件 2，发现部分商品销售量与时间有关，因此使用**皮尔逊相关系数**探索各类商品与时间的相关程度。分析相关系数分布图发现可以将商品划分为**与时间有关**和**与时间无关**的两类，从而采用季节性和非季节性模型预测。对于季节性商品，使用 Holt-Winters、ARIMA、MA 等模型预测，最终调整参数选取使 **MSE 最小**的**指数平滑模型**进行时序预测；对于非季节性商品，使用 KNN、SVM、随机森林预测，最终选取准确率最高为 **91%**的 KNN 预测结果。对于分类问题，先采用**主成分分析**降维处理，通过**累计方差解释图**发现降成 5 维可保留 **90%**的原始信息，分别使用 **Kmeans** 与 **DBSCAN** 模型对商品进行加权分类，通过可视化对比发现 **DBSCAN** 的结果更加准确：**17 类**，结果见表 6。此外，本文还寻找**最佳轮廓系数**对聚类模型进行了**评估**，发现准确度高达 **93%**。

针对问题二：面对商家-仓库-商品维度中新增的信息，结合问题一历史数据进行预测。将问题可划分为：**预测新增商品**、**预测更换仓库的商品**两方面求解。问题一得出：**同类型商品分布特征一致**，因此对于新增商品，只需判断它的类别，使用问题一中同类别的预测方法预测即可。对于更换了仓库的 **1683** 号商品，计算该商品与其他商品的**余弦相似度**，通过数据对比发现**最相似**的同类商品为 **877 号**，而它在问题一中被判定**与时间无关**，因此继续使用 **KNN** 算法进行预测。

针对问题三：参考题目给的去年双十一数据对六月份需求量进行预测，但由于 11 月促销期数据量较少，本文将 12 月份作为 10 月份数据，并与 11 月份的数据一同构成趋势增长的数据；并将该递增数据与 5 月份数据一同作为训练集，预测 6 月份的销售数据。本文基于 **SVM**、**RDF**、**ARIMA**、**指数平滑**四种基础模型和 **GBR** 模型的 **Stacking** 集成学习模型进行预测。对比各模型发现 **Stacking** 集成学习比单一模型预测结果**准确度高**。

关键词：皮尔逊相关系数　指数平滑　DBSCAN　余弦相似度　Stacking 集成学习

目录

一、问题的提出 1
1.1 问题背景 1
1.2 问题重述 1
二、问题的分析 2
2.1 问题一的分析 2
2.2 问题二的分析 2
2.3 问题三的分析 2
三、符号说明 3
四、模型的假设 4
五、数据预处理 4
5.2 数据清洗 5
5.2 数据特征的初步探索 6
5.2.1 皮尔逊相关分析 6
5.2.2 商品信息中类别属性分析 7
六、模型的建立与求解 8
6.1 问题一模型的建立与求解 8
6.1.1 季节性时间序列模型的建立 8
6.1.2 随机森林模型的建立 9
6.1.3 KNN 模型的建立 10
6.1.4 多分类 SVM 模型的建立 10
6.1.5 非季节性预测模型的选择 11
6.1.6 需求量预测结果 12
6.1.7 预测结果合理性分析 13
6.1.8 主成分分析 14
6.1.9 多种分类模型的建立 16
6.1.10 分类结果 16
6.1.11 分类结果合理性分析 16
6.2 问题二的建立与求解 17
6.2.1 问题二模型的建立 18
6.2.2 问题二模型的求解 19
6.3 问题三模型的建立与求解 20
6.3.1 问题三模型的建立 20

6.3.2 问题三模型的求解 22
6.3.3 问题三模型的评估 23
6.4 库存优化问题的设想 24
七、模型的评价与推广 25
7.1 模型的评价 25
7.2 模型的推广 26
参考文献 27
附录 28

一、问题的提出

1.1 问题背景

随着互联网技术的蓬勃发展，电子平台销售模式更新甚至代替了以往的传统的销售模式。电子平台销售模式也称作线上销售，线上销售主要由两方面来实现：第一是电商平台；第二是商家。电商平台相当于管理者，提供一定数量的仓库供各个商家存储商品货物，对外进行销售，它为商家提供销售渠道，并监督和管理负责商家与客户之间的交易情况，通过人员科学的管理手段和计算机的大数据智能分析系统，确保消费者能够获取自己所需及相应的保障服务，确保在法律法规下，商家和客户都得到应有的利益；而商家因为季节变动、节假日价格促销等原因每个商家在各个时间段负责进货并卖出不同类型、不同数量的商品以满足人们的日常生活需求，它相当于利用好电商平台这个媒介，利用大数据分析手段降低库存成本，解决供应链问题并获取自己的利润。电商平台和商家之间的合作关系有助于促进电商行业的健康发展和消费者权益的法律保护。

1.2 问题重述

问题一：使用如下附件 1-4 数据，根据 2022 年 12 月 1 日到 2023 年 5 月 15 日出货量预测出各商家在各仓库的商品在 2023-05-16 至 2023-05-30 的需求量，并对预测的结果进行评价。同时由商家、仓库、商品形成的时间序列使用一定的分类方法分类使得同一类别商品随时间或季节变化的趋势基本一致。

- 问题一依据数据

  附件 1：商家历史出货表

  附件 2：商品信息表

  附件 3：商家信息表

  附件 4：仓库信息表

问题二：由于市场竞争等因素，出现了新的商品种类，因此对于新出现的商家+仓库+商品维度，探讨在出现的预测维度，如何进行 2023-05-16 至 2023-05-30 的需求量的预测及相似序列的分类。

- 问题二新增依据数据

  附件 5：新品历史出货量表

问题三：由于可能有国庆节、双十一、618 狂欢节等节日活动，商家会进行促销活动，加大促销力度。按照常理，许多人会选择在这时候购置家具、购买通讯设备等补贴家用，因此对于利润大、易销售的产品，各个仓库的商品的需求量在节日期间可能有大幅度增加；对于利润低、运输损耗大等产品，商品在仓库的存储量应随之减少。依据附件 6 所给的去年双十一期间商品需求量，预测 2023-6-1 至 6-20 的需求量并完成分类。

- 问题三新增依据数据

  附件 6：促销期间商家出货量表

二、问题的分析

2.1 问题一的分析

对于问题一，要求预测出各商家在各仓库的商品在 2023-05-16 至 2023-05-30 的需求量并且对各个商品进行分类，使得同一类别相似性最高。明晰一个目标：出货量由三方面因素确定，分别为①商家；②仓库；③商品。而它们又有各自的子因素来共同决定各个商品的出货量，具体如图 1 所示。首先，对于预测问题，本文先考虑季节性变化因素，12 月至 5 月份数据可以划分为冬、春、夏三季，抽象理解为随着温度的上升，某商品的需求量呈现三种状态：上升、下降、波动、平稳。于是经过数据预处理后，采用控制变量的思想，统计出同一商家下、同一仓库下各个商品的出货量，使用皮尔逊相关系数得出每个商品与时间的相关性程度，考虑季节性时间序列模型预测和非季节性预测。其次，对于分类问题，首先将附近 1-4 进行拼接，然后商家、仓库、商品三列均相同的数据数据剔除，保留每一种商品的特征量，接着进行标签编码、剔除无关列、归一化等数据预处理，接着进行基于求解累计方差贡献率的主成分分析，然后标准化处理数据，计算权重，分别使用 Kmeans 和 DBSCAN 等其他常用的聚类算法完成分类。

2.2 问题二的分析

对于问题二，出现了新的数据。由于新商品上市或商品更换仓库，需要重新预测新维度下各商家在各仓库的商品在 2023-05-16 至 2023-05-30 的出货量。首先，同问题一一样，将附件 5 数据进行分组，按照时间从小到大排序。将附件 1 和附件 5 进行匹配，得到新增商品和更换仓库的商品。根据题目，由于部分商品的销售时间过短、仓库存在新增或切换等情况，导致该预测维度下的历史数据过少，于是必须需要通过算法找出历史一般规律，即就是找出相似出货量商品的历史数据来扩充所给的新维度数据，从而实现精准预测。其次，这里预测分两步走，一种是预测新增商品，另一种是预测更换仓库的商品。对于新增商品，虽然它的商品编号是新的，但是它的类别可以在商品信息表（附件 2）中查到。因此，将它同类下的其他种类商品历史销量扩充到新增商品数据表格中，通过第一问的聚类结果，已经得知同一类别下的商品分布特征一样。根据商品类别即可判断与时间相关或无关的数据，从而使用对应的预测方法。对于更换了仓库的商品，我们先找出“更换后仓库原有的商品”和“这个商品原本所在的仓库”中的原始出货量数据。使用余弦相似度检验将现有的出货量与所找数据进行匹配，计算相似度。若最相似的商品哪一类，则对新维度数据使用同类型商品的预测方法。

2.3 问题三的分析

对于问题三，新增了 2022 年的双十一商品出货量数据，需要预测 2023 年 6 月 1 日至 6 月 20 号的出货量。首先，进行分组操作，在仓库、商家、商品一致的情况下，查看所给双十一期间各个商品的时间维度是否一致，发现每个商品的数据时间长度在变化并不统一，考虑到所给的 11 月份与需要预测的 6 月份的商品均处于打折期间，所以实际情况下，这两个月份商品的出货量特征较为相似。但是，所给双十一的数据较少，直

接预测的结果极大可能不准确，所以必须增加原始数据量。其次，观察附件 1 发现只有 2022 年 12 月份的数据，考虑出货量在双十一前后两个月份变化幅度不可能很大，同等情况下，前一个月 10 月和 12 月的出货量相较于 11 月份的出货量都少些，故将其作为 2022 年 10 月份数据。可以理解为 2022 年 10 月到 11 月数据呈现递增趋势；用以预测也同样呈现递增趋势的 2023 年 5 月到 6 月份数据，并且根据实际情况，应该将这些商品都分类为与时间相关的商品（可以用时间序列模型预测），最后将 2022 年的 10 月和 11 月份商品出货量数据作为训练集，2023 年的 5 月份总体数据作为测试集，基于支持向量机（SVM）、随机森林（RDF）、ARIMA 时序预测等基础模型，综合使用 stacking 集成学习模型进行预测。所有问题的初步求解思路，如图 1 所示。

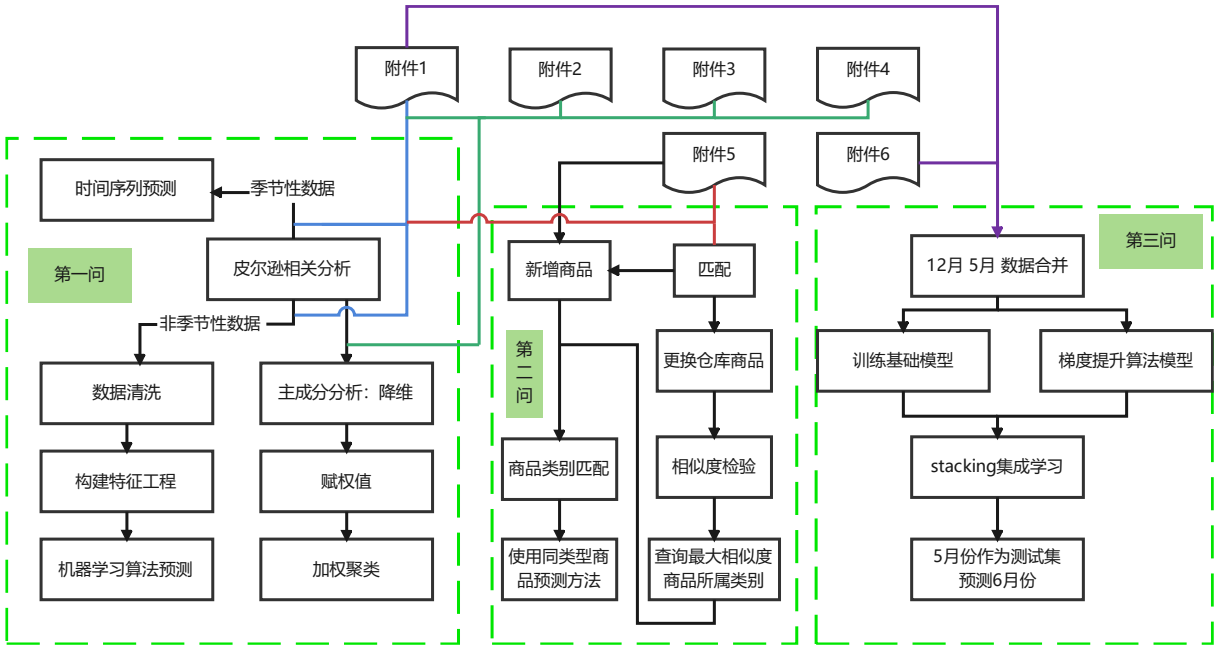


图 1 问题的总汇流程图

三、符号说明

符号	说明
t	时间
W	类别属性 W_i
i	第 i 个商品
j	第 j 个商家
k	第 k 个仓库
W_i	商品所属的第 i 个类别
θ	两个向量之间的夹角
$y(i)$	第 i 个商品的出货量
H_i	商品所属的第 i 类仓库区域
$\hat{y}(i)$	第 i 个商品的预测出货量

四、模型的假设

1. 假设已给商品的出货量数据准确度和稳定性高
2. 假设除个别数据可能存在失误外，其余所有数据真实且符合实际情况
3. 假设各类商品的购货渠道、运输渠道、销售链、产品链等处于稳定状态
4. 假设同一类别的商家、仓库、商品在需求特征上更为相似，因此可以使用相似性度量将它们分类
5. 假设去年双十一期间的需求数据可以用来预测今年六月的需求，即这两个时间段出货量存在很大的关联性
6. 假设需求量在不同地理位置的仓库之间存在差异，地理因素可能影响货物运输和供应链的效率
7. 假设大型促销活动的影响是可测量的，可以使用促销活动的相关因素来解释需求量的变化

五、数据预处理

5.1 数据清洗

观察附件 1 所给商品的历史销售信息，发现总计大约 33 万数据，数据量很多表明极大可能存在异常值。因此，本文先对数据进行清洗，主要分为以下几个方面。

Step1 剔除重复值

处理附件 1 至附件 6 所有数据，发现除附件 4 外，其余所有附件均无重复值，附件 4 的重复值如表 1 所示。

表 1 附件 4 重复数据

所在行数	warehouse_no	warehouse _category	warehouse _region	重复个数
23	wh_29	区域仓	华东	1
24	wh_29	区域仓	华东	
32	wh_38	区域仓	西南	1
33	wh_38	区域仓	西南	

Step2 标签编码

使用 Jupyter 编译器中 python 语言的一个内置函数 LabelEncoder 函数进行编码，从而将文本型数据转换为整数数值型标签。对于附件 4 仓库信息表，将原始表格中的仓库类别中心仓、区域仓变为 0 或 1；七个仓库区域从 0-6 编码；商家类别从 0-15 编码；商家规模从 0-3 编码；商品一级分类从 0-19 编码，二级分类从 0-61 编码；三级分类从 0-209 编码。以上编码均为整数类型编码。总计对八列数据进行编码，编码后的表见支撑材料（拼接 1-4 表 1.5）。

Step3 异常值处理

这里，考虑到大数据量，并且数据分布规律还不是很明显，不能轻易随意认为哪些数据为异常数据，因此，本文选择采用箱线图和小提琴图来识别异常值。原因是：它们结合起来可以非常直观的展示大量数据的分布特征极其概率密度，比如：通过图形，可以看出这些数据的中位数、最大值、四分位数、四分位距等信息量，而它们以四分位距为临界线，即将数据点在中位数 25%以外的数据认为是异常值，及时 25%以外的数据偏离数据中心足够远，也不会影响后期的数据处理，因此方法很合理。

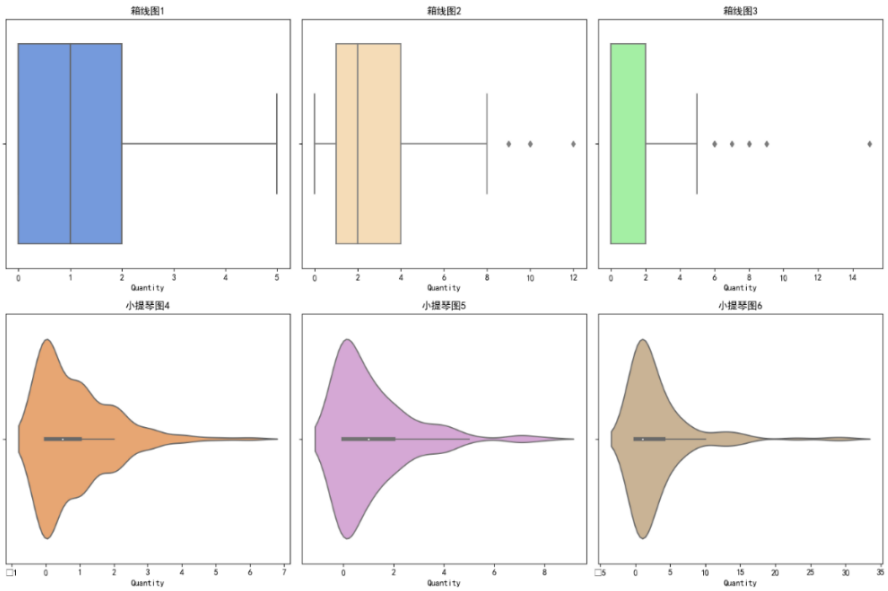


图 1 小提琴图和箱线图

Step4 训练集与测试集划分

对于三个问题，后期对于预测和分类问题，面对庞大的数据量，本文会优先选择随机森林、决策树等机器学习算法构建基于特征工程量化指标体系，因此后续极有可能需要对训练集、测试集数据进行划分，对于所有的训练集和测试集数据，我们大多数情况下将它们的比例设置为 4：1，后面模型将会详细解释设置该比例的原因。如果这个比例下效果不好，则调整参数使得模型建立的更好，使用 sklearn 库中的一个名为 train_test_split 的函数用以分割训练集和测试集，并且设置随机数种子 random_state 用以随时获得新的训练集和测试集的划分，寻找最优模型。

Step5 数据标准化

因为数据标准化在一定程度上可以提高算法消除原有数据中的冗余度，降低原始数据中的“鸿沟差异”，使得数据整体质量提升，特别适合用于处理大数据量问题的研究，从而使得后期模型的建立与求解更快且更准确。本文使用 $Z-score$ 方法对数据进行标准化处理，主要用于后文进行主成分分析时，数据处理的基础操作。用 $\bar{x}$ 表示某一列数据的平均值， std 表示该列数据的标准差。则计算公式表示如下。

$$\begin{cases} std = \sqrt{\dfrac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^2} \\ \bar{x} = \dfrac{1}{n}\sum_{i=1}^{n}x_i \end{cases} \tag{1}$$

数据预处理中的其中一个主要环节：数据清洗，详细数据处理操作汇总如图 2 所示。

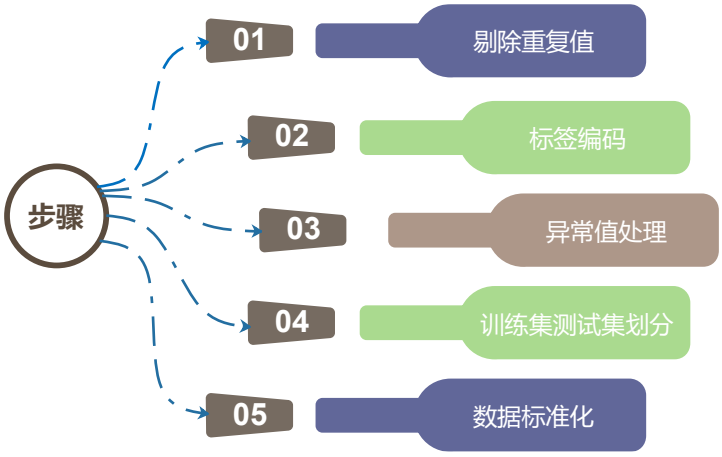


图 2 数据预处理关键步骤

5.2 数据特征的初步探索

经过商品信息的初步观察，发现部分商品的销售量与时间有很大的关系，举个很明显的例子：附件 2 中，对于“驱蚊驱虫”这类商品，根据生活常识，更可能用于夏季天气比较暖或者很热的情况下，冬天大多情况下人们不需要购置这个商品。因此可以粗略认为它的销量是与时间的分布有关的，随着气候逐渐变暖，出货量越来越大；而对于“办公设备”这类型商品，常年四季都可能随时销售，因此可能随时间变化并不明显。因此，本文先使用相关性分析方法探索各种商品出货量与时间的相关性大小及分布特征。

5.2.1 皮尔逊相关分析

皮尔逊相关系数(Pearson correlation)是用来反应两个变量之间的线性相关程度，而商品出货量和时间数据表示两个维度下的变量，可以尝试用皮尔逊相关分析来衡量它们之间的相关性。不妨用 $\rho(x,y)$ 来表示，其中 x 和 y 分别表示两个随机变量，计算公式如(2)所示。

$$\rho(x,y)=\frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i-\bar{y})^2}} \tag{2}$$

令 $y=t$，即用 y 这个变量表示时间 t。明显 $\rho(x,y)\in[-1,1]$。若 x,y 分别表示出货量和时间，那么当时间逐渐增大时，另一个变量商品的出货量也随之增大，则表明这两个变量之间是正相关的，也就是说该商品的出货量与时间相关，相关系数大于 0，并且相关程度越强，则越趋近于 1。代表该商品的季节性特征越明显。如果时间逐渐增大时，而商品的出货量却在减小，则表明两者为负相关（该商品的出货量与时间仍然相关），相关系数小于 0，且相关性越强，越趋近于-1。如果相关系数等于 0，则表明它们之间不存在线性关系。通常的相关程度等级划分，如表 2 所示。

表 2 相关性程度等级表

相关程度	极强相关	强相关	中等相关	弱相关	无相关
$\lvert\rho(x,y)\rvert$	(0.8,1]	(0.6,0.8]	(0.4,0.6]	(0.2,0.4]	(0,0.2]

计算每种商品出货量 x 与时间 t 之间的皮尔逊相关系数，并绘制相关系数大小特征分布图查看商品的时间属性信息。具体分布如图 3 所示。

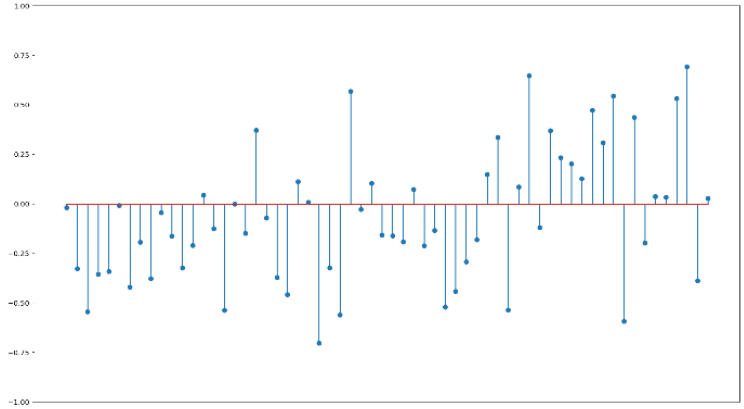


图 3 各类商品与时间的相关性分布图

从图 3 可以看出，不同种类的商品出货量与时间的相关性不同，它的变化幅度大致在[-0.75,0.75]之间，有的相关系数特别小，甚至趋于 0；有的相关系数特别大，说明出货量与季节关联密切。所以，商品可以分为两大类：①销售量与时间相关的商品；②销售量与时间无关的商品。故为使得后期问题处理更加方便，本文将所有商品划分为两类：季节性数据和非季节性数据，完成后期的预测和分类。

为了表示哪些商品与时间的相关性较强，将季节性与非季节性划分的临界点设为 0.6，即当本题使用 Pearson 相关分析出来的某些商品相关性大于 0.6 时，则认为该商品的出货量呈现季节性变化，后期使用时间序列模型进行预测；反之，使用其他类型的模型预测非季节性数据。经过对附近一的处理，筛选出季节性商品和非季节性商品，如表 3 所示。其余数据见支撑材料中与时间有关的商品表 1.2 和与时间无关的商品表 1.3。

表 3 季节性和非季节性商品原始数据划分

编码属性	与时间相关的商品			与时间无关的商品		
seller_no	seller_11	seller_11	seller_11	seller_10	seller_10	seller_10
product_no	126	140	143	1914	1915	1916
warehouse_no	wh_1	wh_1	wh_1	wh_1	wh_1	wh_1
	. . .	. . .	. . .	...	...	...
product_no	705	711	708	1764	96	115

5.2.2 商品信息中类别属性分析

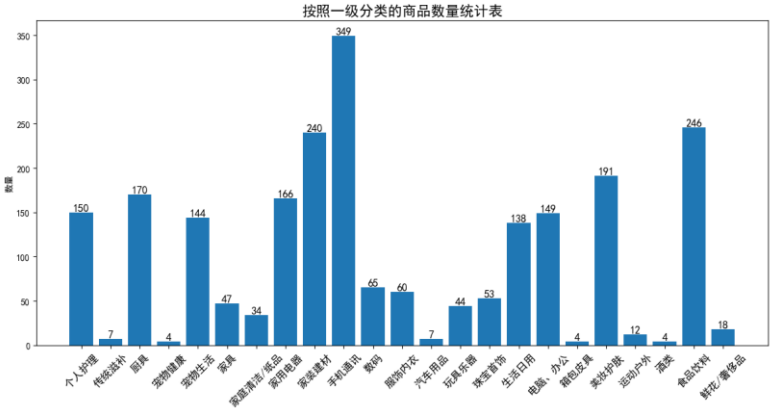


图 4 商品一级分类图

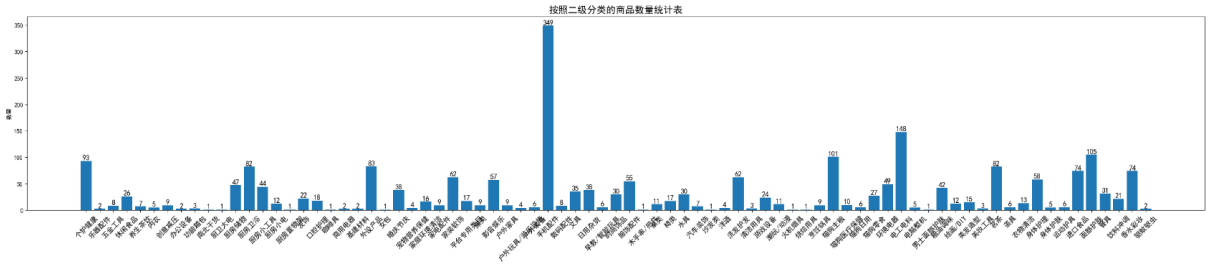


图 5 商品二级分类图

已知，商品被分为：一类、二类、三类这几个类别。根据附件 2 商品信息表，分别绘制出在各个商品类别下各类商品的出货量，由于篇幅限制，这里只展示一级分类、二级分类销量分布图。通过图 5，观察到不同商品的销售量不同，而图 5 比图 4 明显，商品划分的更加细致，表明很多种类的商品出货量是不同的，另外将上述两个图与三级分类的数量图做对比，发现三级分类对商品划分的更加细致，但是观察原始表格，三级分类是对二级分类的很小程度上的进一步划分，比如：二级分类中分为进口食品，三级中则分为进口食品_1、进口食品_5 等等，可见对于商品的类别属性，二级分类已经足够，并且能保留绝大部分信息。因此，本文后续基于二级分类进行问题的研究。

六、模型的建立与求解

6.1 问题一模型的建立与求解

对于问题一的第一小问，很明显的预测类问题，本文考虑到时间因素，基于文章前面所述，将商品划分为：季节性数据、非季节性数据。最终使用时序预测和非时序预测。

6.1.1 季节性时间序列模型的建立

MA (Moving Average, MA) 也称移动平均模型，是用一组最近的实际数据值来得到未来一期或一段时间内数据的预测值的一种常用方法。当时间序列的数值受周期波动或者不规则变动的影响，起伏较大，不易显示未来趋势时，为避免多重因素影响，则可以使用移动平均法。

ARIMA 也称自回归移动平均模型，相当于 MA 模型的升级，它是用来预测平稳数据序列的，意味着数据的均值和方法不随着时间变化而变化均为常数，数据总体呈现平稳特征，适合本题季节性模型中平稳变化的数据预测。该模型的一般形式如公式所示。AR（自回归项）、MA（移动平均项）在公式(3)中分别代表 p 和 q，对于平稳序列需要确定参数 p 和 q；对于非平稳需要先进行 d 阶差分，将其转化为平稳序列。

$$X_t = c + \alpha_1 X_{t-1} + \alpha_2 X_{t-2} + \ldots + \alpha_p X_{t-p} + \varepsilon_t + \beta_1 \varepsilon_{t-1} + \ldots + \beta_q \varepsilon_{t-q} \tag{3}$$

指数平滑模型通常有一次平滑、二次平滑，甚至三次平滑。它作为经典的一种时序预测方法，根据实际生活经验，若我们预测明天的天气状况，如果前一天的天气特别阴暗，且前三天的天气都比较阴暗，那么明天极大可能处于阴天；同理，当我们想要知道未来两周该商家的销售量时，则最近几周这个商家的店铺销售状况对预测结果有着很大的参考价值。而指数平滑模型则是利用这个思想去进行数理分析，实现预测。它的具体

计算公式如下。其中α为平滑系数，这个数值越准确，预测效果越好。

$$\begin{cases} S_t^{(1)} = \alpha X_t + (1-\alpha) S_{t-1}^{(1)} \\ S_t^{(2)} = \alpha S_t^{(1)} + (1-\alpha) S_{t-1}^{(2)} \\ S_t^{(3)} = \alpha S_t^{(2)} + (1-\alpha) S_{t-1}^{(3)} \end{cases} \tag{4}$$

Holt-Winters 三参数指数平滑法是[4]由温特斯（Peter R.Winters）通过添加季节性构成季节性指数，在 Holt 双参数指数平滑法拓展而来。在处理具有趋势和季节性的时间序列数据时，三参数指数平滑法通过修改 Holt 指数平滑法来使用。具体来说，它使用三个参数来分别估计水平、趋势和季节性分量。这里，本文猜想数据有增长或减少的趋势，因此建立此模型见公式t为当前期；h为预测超前期数，也称之为预测步长；x_t第t期得实际观测值；b_t为t时刻的预测趋势；α是水平平滑参数；β为趋势平滑参数。其中，重要的是季节性分量则反映时间序列的周期性变化。适用于问题一的周期性变化的商品出货量预测。

$$\begin{cases} l_t = \alpha x_t + (1-\alpha)(l_{t-1} + b_{t-1}) (\text{水平平滑方程}) \\ b_t = \beta(l_t - l_{t-1}) + (1-\beta) b_{t-1} (\text{趋势平滑方程}) \\ \hat{x}_{t+h} = l_t + h b_t (h = 1, 2...) (\text{预测方程}) \end{cases} \tag{5}$$

6.1.2 KNN 模型的建立

KNN（K-Nearest Neighbor)，也经常被称为 **K 近邻**，属于常用的 ML（Machine Learning）算法中一类有监督学习算法，适用于分类和归回问题的研究。该算法的核心思想是：距离该样本中心最近的 K 个点中大多数点所属的类别标签，即就是这个样本的类别。本文问题一所使用的 KNN 算法简单示意图如下。

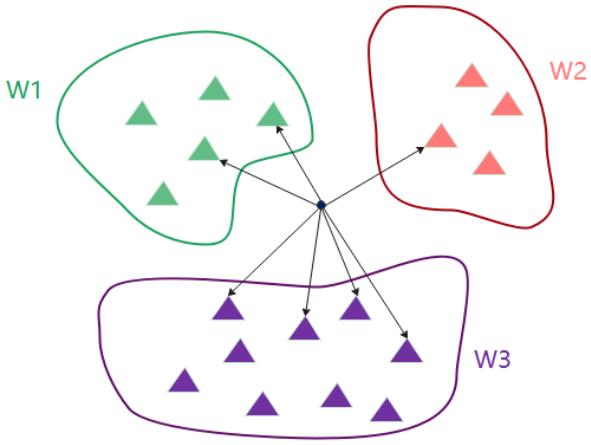


图 6 KNN 算法示意图

如图 6 所示，W1、W2、W3 分别代表训练集中的三个类别。其中，与聚类中心距离最近的 7 个点（K=7）如图中箭头所指。从示意图可以很明显看出，与其最相近的 7 个点中数量最多的点所处的类别为 W3。

6.1.3 随机森林模型的建立

随机森林 (Random Forest) 是一个[1]由多棵决策树形成的综合性学习模型。它结合

了两种算法，一类是决策树；一类是 Bagging 集成学习方法。当研究分类问题时，测试数据会被送到每一棵决策树中进行预测，接着每一棵决策树进行投票，票数最多的类别属性为最终分类结果；而当研究回归问题时，它的预测输出结果为所有决策树输出的均值。

如若每次都用完整的训练样本集训练树，那么每棵树都是相同的，因此，随机森林通过 Bootstrap[2]有放回式随机抽样得到训练样本，然后循环依次训练每棵决策树。巧妙利用随机抽样的特性，在一定程度上消除过拟合现象。并且，在训练决策树时，每个节点所用的特征也是随机抽样得到的，即从特征空间中无放回式随机抽取 k 个特征参与训练，构成随机特征集。因此，随机森林进行了特征值和样本集两方面的随机采样。具体流程图，如图 7 所示。

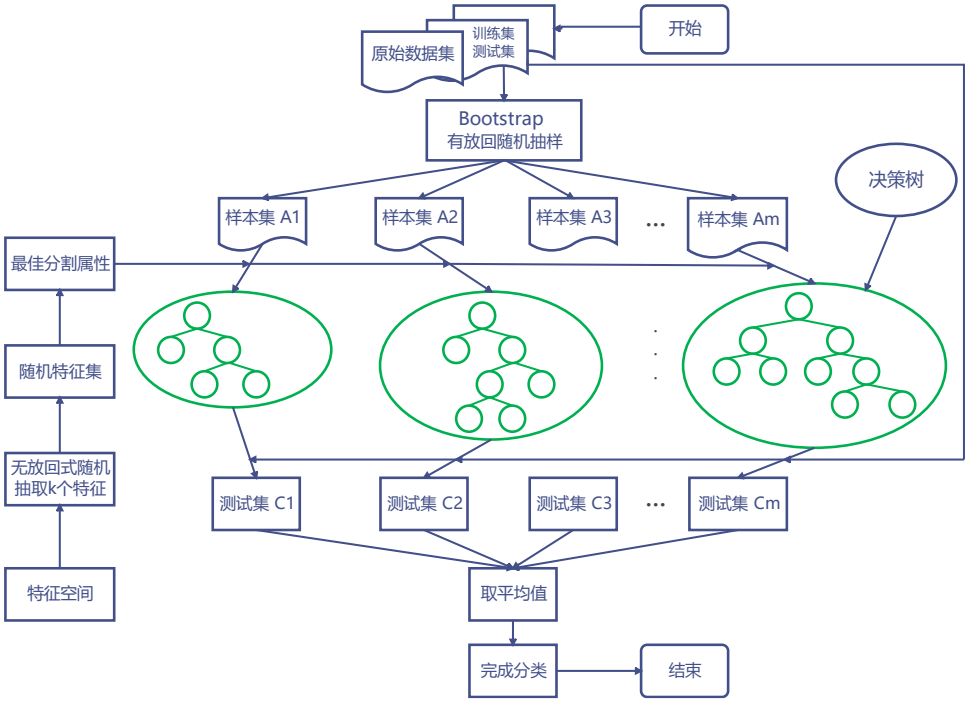


图 7 随机森林流程图

6.1.4 多分类 SVM 模型的建立

SVM (support vector machines) 是 ML 中的一种监督学习[3]算法，是一种二分类模型，SVM 模型是定义在特征空间中间隔最大的线性分类器。同 KNN 算法一样也适用于分类和回归问题的研究。该模型核心目标是寻找一个最佳的超平面，将所有的数据点分离开。但是这个超平面能最大化分开数据类别，使得距离这个超平面最近的那些数据之间的距离最大，分类效果更明显。

由于 SVM 一般常用于解决二分类问题，但是本文需要分类多个类别，所以需要适当地进行变化。本文地多分类向量机通过组合多个二分类模型 SVM 来实现，**OvR**：One VS Rest，用二分类算法来解决多分类问题的一种策略。它的核心思想就是一对多（剩余）。算法流程是如果需要对 n 种类别的样本进行分类，则取一种样本作为一类，将所有剩余的样本看做另一类，这样问题转化为求解 n 个二分类问题,具体算法如图 8 所示。

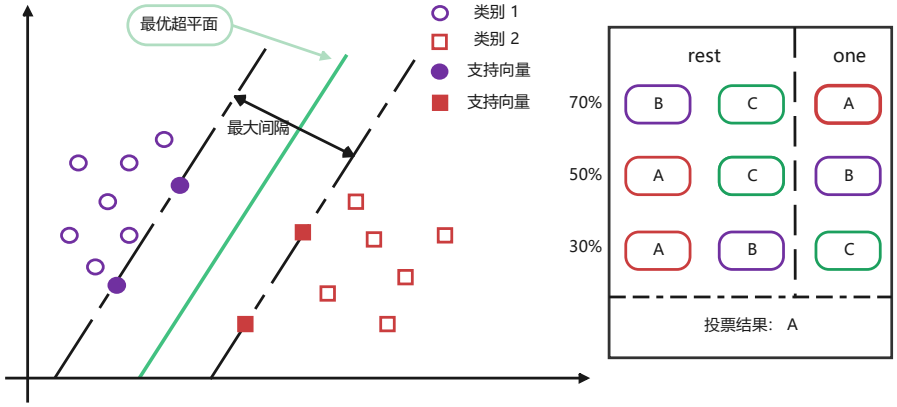


图 8 多分类 SVM 模型的建立

若数据线性可分，则线性 SVM 的决策函数可以表示为，如公式(6)所示，其中函数只有三种结果 $f(x)=1$； $f(x)=0$ 和 $f(x)=-1$，若数据线性不可分，需要使用核函数将原始数据映射到更高维度的空间来进行分类，本文经过 matlab 软件初步探索了数据特征，发现大部分数据有正态分布的特征，所以选用高斯函数作为它的核函数，如公式所示。

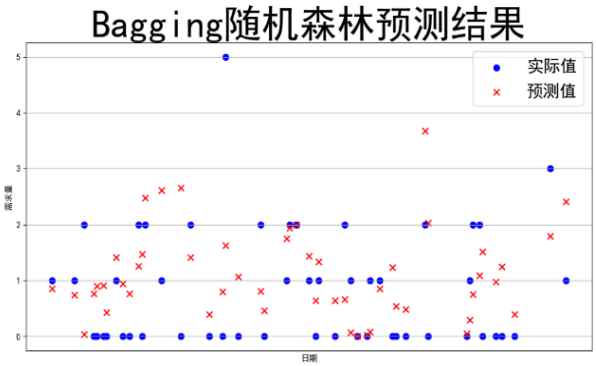
$$f(x)=sign(\omega \cdot x+b) \tag{6}$$

$$Gaussian \quad Kernel \quad K(X_1,X_2)=\exp(-\gamma \| X_1-X_2 \|^2) \tag{7}$$

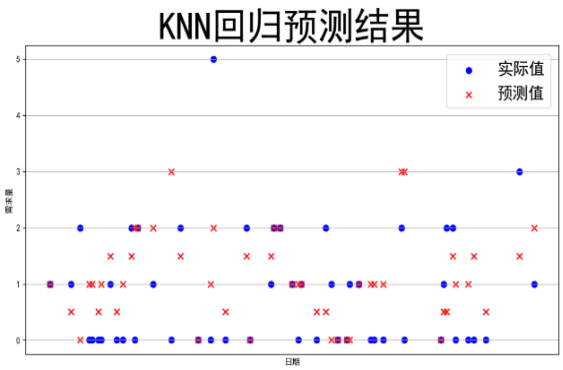
高斯核函数的函数值，可以反映出两个样本点之间的相似度大小，当参数 γ 一旦确定，那么两点间的距离越小，则相似度无限趋近为 1；若两点间的距离越大，则相似度无限趋近为 0，从而循环遍历每个样本点，最终完成分类。

6.1.5 非季节性预测模型的选择

分别使用随机森林、KNN、SVM 预测的散点效果图如下，可以大致看出 KNN 的拟合度更高。分别计算出各自的 MSE，得出 KNN 的 MSE（均方误差）最小，且三者的拟合精度排序为 KNN > SVM > 大于 Bagging。于是问题一选择 KNN 算法进行非季节性出货量的预测。具体如图 9 所示。

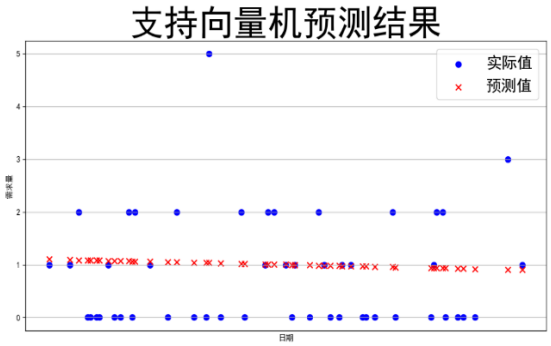


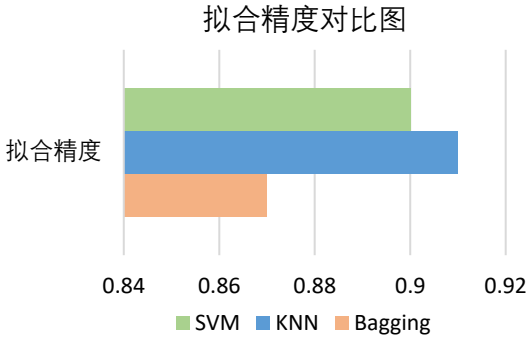
(1) Bagging 随机森林预测效果图



(2) KNN 回归预测效果图

观察子图(1)，Bagging 随机森林预测效果图中，预测值与实际值的点位分布范围一致，且重合度较高，只是有部分数据预测不准确，观察子图(2)，KNN 回归预测效果图中，预测值与实际值的点位分布范围一致，且重合度相比较子图(1)来说，更高，极少数据预测不准确。





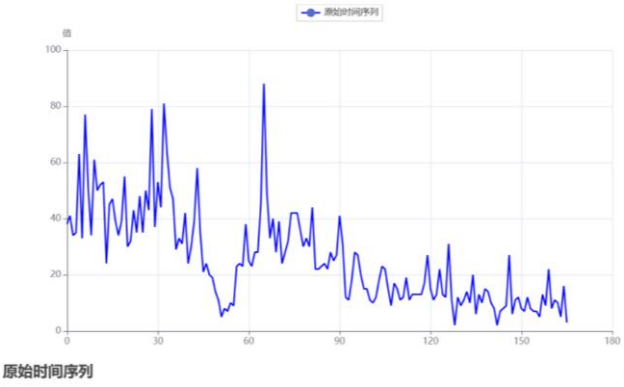
（3）支持向量机预测效果图　　（4）三种算法效果对比图

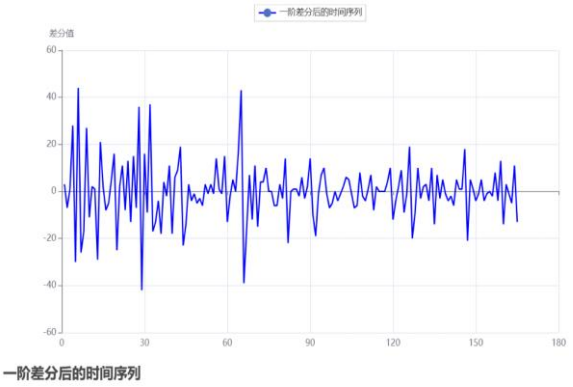
图 9 各类预测方法效果对比图

6.1.6 需求量预测结果

a. 季节性时间序列模型预测出货量结果

这里先进行 ARIMA 模型的实现步骤。ARIMA 模型专门处理平稳性时间序列问题，对于非平稳性时间序列的问题，必须首先将其转化为平稳性数据，才能进行预测。对此，我们必须先检验原始训练数据的平稳性，必要时，要对数据进行差分操作或反差分操作。在程序中，计算出当差分系数为一阶时，即 $d=1$ 时效果最好，如图 10 所示。

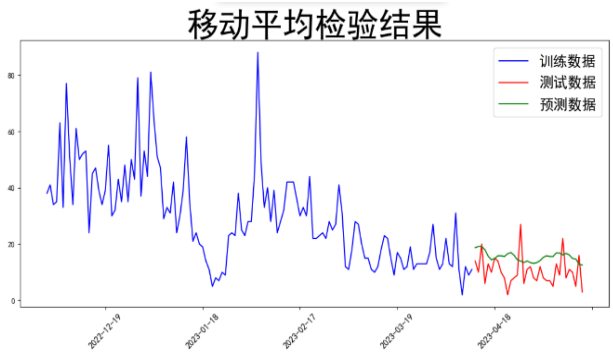


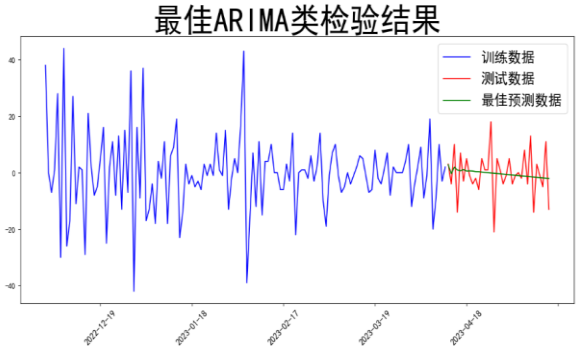


（1）原始时间序列图　　（2）一阶差分后的时间序列图

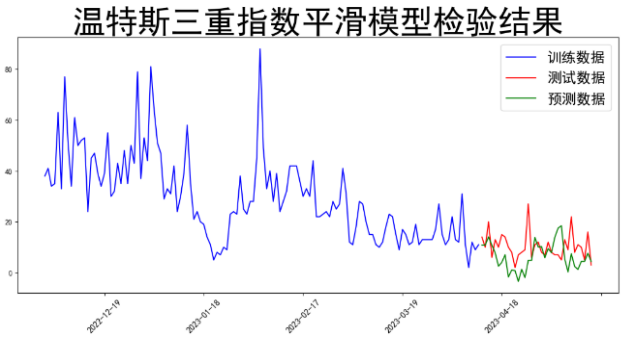
图 10 平稳时间序列的转化

模型建立过程中，一部分商品的出货量是随着季节（时间）变化的，故本文建立了多个季节性时间序列预测模型。各个模型的预测结果图 11 所示。

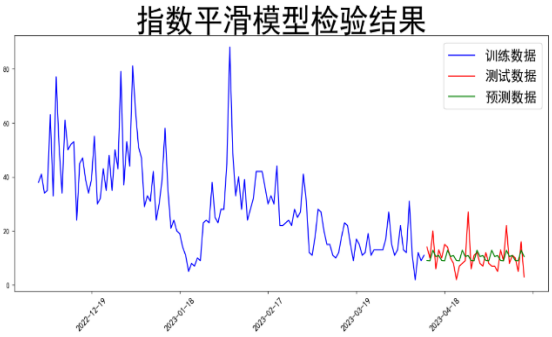




（1）移动平均预测结果图　　（2）最佳 ARIMA 预测结果图



（3）温斯特三重指数平滑预测结果图



（4）指数平滑预测结果图

图 11 各个模型的预测结果

从上图中可以看出，三次指数平滑模型的预测效果最好，预测数据的大体趋势、起伏与所使用的测试数据一致，两者数据值差异并不是很大。所以最终选择三次指数平滑模型进行第一问需求量的预测。详细出货量预测结果见表 3。

表 3 三次指数平滑模型出货量预测结果

seller_no	product_no	warehouse_no	date	qty
seller_11	product_126	wh_1	2023-05-16	4
seller_11	product_126	wh_1	2023-05-17	4
seller_11	product_126	wh_1	2023-05-18	8
seller_11	product_126	wh_1	2023-05-19	5
seller_11	product_126	wh_1	2023-05-20	6
...	...	...	...	...

b. 非季节性模型预测出货量结果

分别随机森林、KNN、SVM 三种算法，由于 KNN 的拟合精度最高，精度达到了 91%，而随机森林的拟合精度只有 87%，SVM 为 90%。所以最后选择 KNN 模型进行预测，结果如下表所示。

表 4 KNN 算法出货量预测结果

seller_no	product_no	warehouse_no	date	qty
seller_10	product_1914	wh_1	2023-05-16	0
seller_10	product_1914	wh_1	2023-05-17	0
seller_10	product_1914	wh_1	2023-05-18	0
seller_10	product_1914	wh_1	2023-05-19	0
seller_10	product_1914	wh_1	2023-05-20	0
...	...	...	...	...

6.1.7 预测结果合理性分析

- 寻找最小 MSE

在使用指数平滑模型预测非季节性数据时，对指数平滑模型中参数的选取进行了探索，寻找最小 MSE 值和对应的参数，用最优参数训练了指数平滑模型。

这里所指的参数是指数平滑模型中的季节性周期，绘制 MSE 的大小随参数值的变化趋势图。最终发现，当参数选为 50 的时候，MSE 是最小的，因此代码中的最优参数应该设置为 50。具体 MSE 随季节性指数平滑模型参数的变化图，如图 12 所示。



图 12 寻找最优 MSE

6.1.8 主成分分析

对于问题一的第二小问，为很明显的分类问题。为了完成分类，本文先进行附件一到附件四的拼接，结果见（拼接表 1.4）。若要分类，则需要根据商品的特征来进行划分，因此需要构建特征工程，但是统计表 1.4，发现总共有 13 列特征量，直接处理不易分析问题，所以必须对现有数据进行降维处理。因此，进行分类前，先采用主成分分析 **PCA(Principal Components Analysis)**，具体分为以下几个步骤。

Step1 剔除无关列

因为需要分类数据，必须提取数据的特征，而 **seller_no**、**product_no**、**warehouse_no** 和 **qty** 这四列数据不能作为划分依据，所以不能将其列入特征工程中。对编码后的数据表进行上述四列数据的剔除。

Step2 数据特征探索

删除四列数据后，剩余九列数据（包含问题一所求的各类商品时间相关系数具体数值），通过表 1.4 中数据，绘制除最后一列时间相关系数外其余八列数据的词云图。如下图所示。

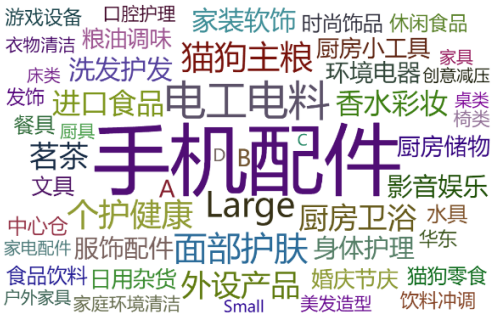


图 13 各种商品词云图

通过图 13 可以看出，“**手机配件**”的标签最大，其次还有“**电工电料**”、“**面部护肤**”、“**Large**”等标签显示也很大；反观之，“**桌类**”、“**华东**”、“**small**”等字眼非常小。词云图中标签的大小反映了两个信息量：①各个商品出现频次的高低映射了商品出货量、仓库分布的不一致；②商品类别、商家类别、仓库类别等各个指标的重要性不一样，对最后分类结果的影响程度不一样。若该标签越大，则说明该特征对分类结果贡献值更大。

因此，不仅需要构造特征工程，还需要得到对应每列指标的权值大小，即又利用到了主成分分析。

Step3 指标归一化

对剩余的八列数据进行 PCA 处理，将这八列相关的指标用几组线性不相关的新变量表示，使得变量个数变少，同时解释了原始信息。为此，先拟合 PCA 模型，获取每个主成分的方差贡献率，接着计算累计方差贡献率，最终绘制累计方差贡献率曲线来判断应该将 PCA 模型降到几维。

此时，各项指标数据的量级之间相差较大，所以不能直接进行分类。首先，先使用本文第五步数据预处理中数据标准化环节来消除量纲之间的影响。其次，为确保主成分分析的科学性与准确性，进一步对数据进行归一化操作。这里本文使用 pytorch 中的 Std_transforms 归一化操作，计算公式如下,其中 *mean* 表示均值， *std* 表示标准差。

$$x = \frac{(x - mean)}{std} \tag{8}$$

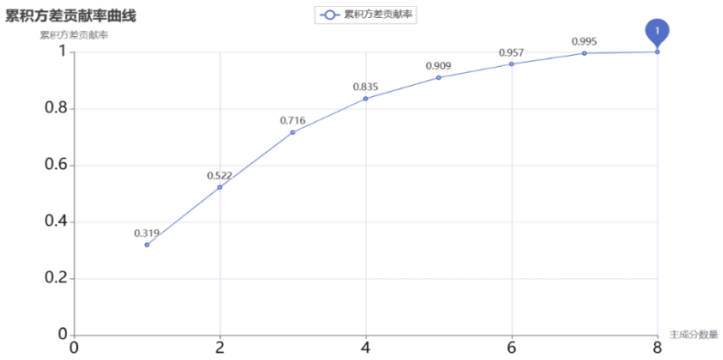


图 14 累计方差贡献率曲线

通过图 14，可以明显看出当数据维度降为 **5** 时，贡献率达到 **90%**，此时能保留原始数据的最大信息量。进行 PCA 处理之后，得到五个线性组合的向量，对每一列求和作为权重。接着对所得权重使用 $Z-score$ 标准化方法，计算公式如所示。其中 $\bar{x}$ 为某一列数据的平均值，

最终经过计算公式(8)归一化后得到的各个特征列的权重大小及分布，如图 15 所示。

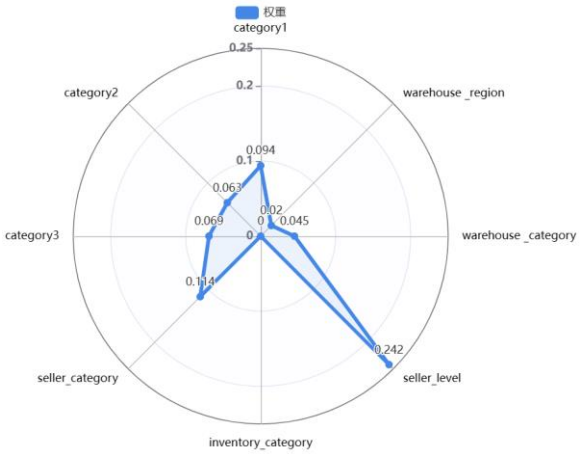


图 15 各个特征列权值雷达图

6.1.9 多种分类模型的建立

K-means 也称 K 均值聚类，它是人为确定的类别数量（簇的数量），使用起来比较简单，收敛速度快，本次聚类过程，如图 16 所示。

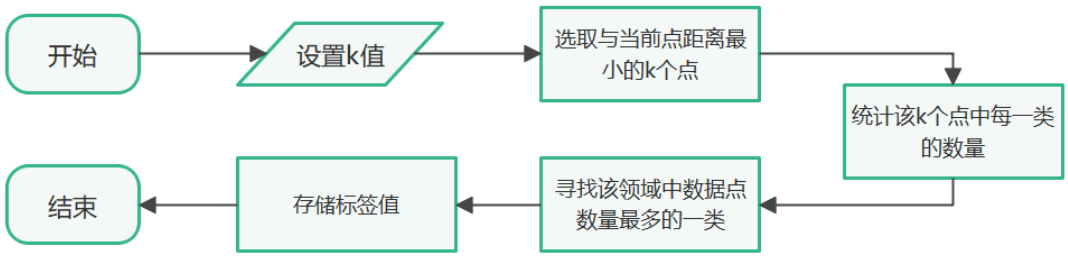


图 16 K-means 算法流程图

DBSCAN 属于无监督的 ML 聚类算法[5]之一。无监督就是它不需要预先标注数据，而是自主学习数据的内部规律来对新的数据进行聚类。相较于 kmeans 聚类算法本身依赖于簇中心以及平均距离衡量数据类别的局限性，只能形成球状的簇并且只能在凸集中聚类，非凸集效果不佳。而 DBSCAN 算法根据样本密度划分类，更适用于无规则数据分类。其算法的伪代码如下所示。

表 5 DBSCAN 算法伪代码

算法：DBSCAN，一种基于密度的聚类算法
输入：数据集 D，半径参数 epsilon，密度阈值 minPts
输出：聚类结果 C
1：初始化所有的簇标签为 unclassified (表示未分类)
2：循环遍历各个样本点 p：
3：　　对于每个 p，找到以 epsilon 为半径、p 中心的邻域内数据集合 Mp
4：　　若 Mp 小于 minPts，则将其标记为 unclassified
5：　　否则，将 p 直接标记为新的簇标签 C_new
6：　　对于 p 的每个邻域点 m
7：　　　　如果 m 的簇标签为 unclassified 且 Mp 大于 minPts，则将 m 也标记为 C_new
8：　　　　将 m 的邻域集合中没有被访问的点加入到 Mp 中
9：返回 C

6.1.10 分类结果

考虑商家、仓库、商品三方面因素形成的时间序列，使用 W-DBSCAN(加权 DBSCAN)算法得到的分类结果展示部分，如下表所示。具体分为了 17 类，详细结果见分类结果表 6。

表 6 分类结果

类别标签	商品个数	商品种类	类别标签	商品个数	商品种类
-1	58	product_1250 product_1257 ...	0	260	product_1914 product_1915 ...
1	285	product_1604 product_1834	2	29	product_1200 product_1206

		...			...
		product_546			product_1938
14	34	product_568	15	9	product_590
		...			...

6.1.11 分类结果合理性分析

- 聚类结果评估

为了使得 W-DBSCAN 聚类算法更具合理性，轮廓系数是用于聚类效果评估的一个重要指标，它的数值越接近 1 效果越好。它用来判断聚类结果的内聚度和分离度，轮廓系数判断样本与其所属簇内其他样本的相似度以及与最近的其他簇样本的不相似度。如果一个被测数据与自己所属的簇其他样本点的距离很小，同时与其他簇中样本点的距离很大，则说明这个样本点所在的簇内内聚度高，簇间分离度大，那么其轮廓系数越大。

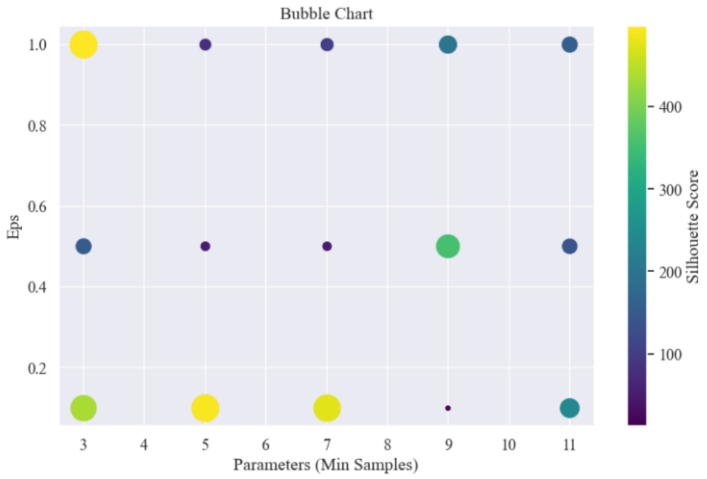


图 17 轮廓系数气泡图

上面图 17 表示在参数 Min Samples=7，Eps=1 时，此时气泡图颜色最深（程序里面已设置如果颜色越深，代表参数越大），代表轮廓系数最大，此时代码运行的结果显示轮廓系数准高达 0.94，非常趋近于 1，说明了使用 W-DBSCAN 分类效果很好。

- 效果图可视化

如下图所示，是使用 W-DBCAN 算法聚类的 17 类的效果图，使用 pycharm 软件构图，发现聚类效果良好，绘制效果图如下。

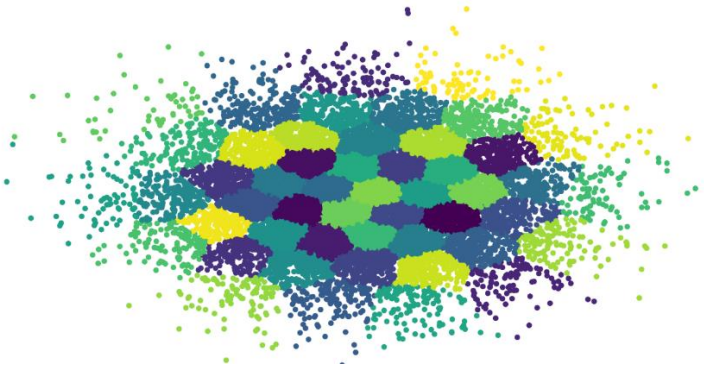


图 18 DBCAN 算法聚类效果图

6.2 问题二模型的建立与求解

- **数据处理过程**

问题二模型的建立需要先对附件 5 做如下处理：

第一步：将所有新增数据按照时间进行排序。具体结果见（时间排序表 2.1）。

第二步：将表 1 和表 2.1 进行数据匹配，若商家和商品两个特征同时匹配，则说明这些商品更换了仓库；如果没有，那么就是新增的商品。从而从附件 5 中抽离出新增商品、更换仓库的商品数据。具体结果见（新维度表 2.2）。

第三步：分两类情况预测

1. 新增商品预测

通过第一问的聚类结果，可以看作同一类别下的商品分布特征一样。根据商品类别判断与时间相关或无关分别是哪些商品。找到与新产品相似性最高的一种商品，因为它的历史数据量大，所以将之前历史数据与新商品的现有数据拼接起来，增大原始数据量。由于附件 2 商品信息中含有新增商品，所以根据商品类别划分与时间相关的商品（与时间有关的商品表 2.3）和与时间无关的商品（与时间无关的商品表 2.4）。

2．更换仓库的商品预测

对于更换了仓库的商品，我们先找出“更换后仓库原有的商品”和“这个商品原本所在的仓库”中的所有原始数据。使用协同过滤算法将现有的出货量与所找数据进行匹配，计算相似度。找到最相似的商品历史数据拼接到新增表格。通过查找，我们发现附件 5 中只有一个商品更换了仓库，为编码为 product_1683 的商品，在原商品信息中，编号为 product_1683 的商品存在，且只需要查询该商品在第一问被分为哪一类，则知道用什么方法进行预测。

6.2.1 问题二模型的建立

同问题一对季节性商品出货量和非季节性商品出货量的预测方法一致，选取模型一中最优的方法，对季节性数据采用指数平滑模型；非季节性数据采用 KNN 模型预测。

动态时间规整模型(DTW,Dynamic time warping)，是使用特殊处理方法寻找两列时间序列中的最佳匹配数据，使用该算法进行相似度匹配，发现效果并不是很好。因此采用**余弦相似度检验**，计算相似度（如图 19 所示）。找到最相似的商品历史数据拼接到新增表格，相当于对新维度商品数据进行扩充，使用最相似的商品所属类别的预测方法，预测新数据未来几周的出货量。计算公式如下

$$similarity = \cos\theta = \frac{qty_i \bullet qty_j}{|qty_i| \times |qty_j|} \tag{9}$$

其中，qty_i、qty_j 分别表示两类商品的出货量，在公式中意味着两个向量，θ 意味着两个向量之间的夹角，它的范围为$[0,\pi]$，所以相似度 *similarity* 的取值范围为$[-1,1]$。1 代表两种商品特征最相似，-1 则代表最不相似。通过上述表达式计算后，得出与编码为 product_1683 商品与其余商品的相似度。如下图所示。通过图中可以发现最大相似度 0.61 为商品 877 号。则将附件 1 中 877 号商品的历史销售量与之前划分出的更换仓库的商品

数据进行组合，来预测更换仓库的商品在未来两周的预测量。

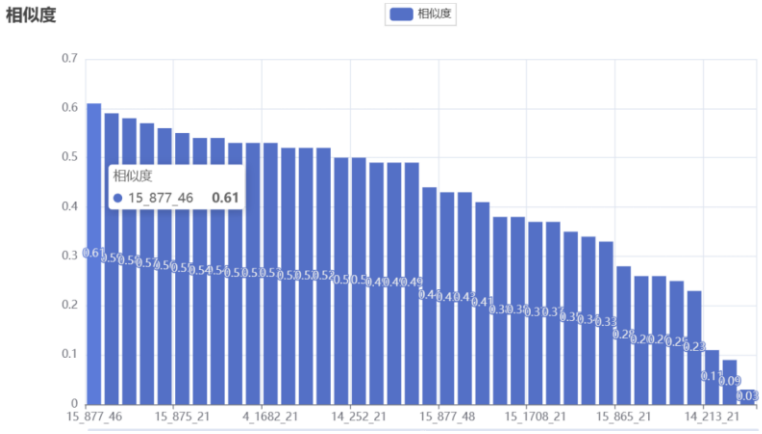


图 19 商品间的余弦相似度

6.2.2 问题二模型的求解

① 新增商品预测结果

通过时间序列模型中的指数平滑法预测与时间相关的商品在未来两周的出货量，部分预测数据如表 7 所示，全部预测结果见（预测结果表 2.5）。

表 7 季节性商品出货量预测结果

	seller_no	product_no	warehouse_no	date	qty
指数平滑法	seller_12	product_2105	wh_1	2023-05-16	10.9493413
	seller_12	product_2105	wh_1	2023-05-17	8.316635139
	seller_12	product_2105	wh_1	2023-05-18	10.35059574
	seller_12	product_2105	wh_1	2023-05-19	12.75550381
	...	...	...	...	...

通过 KNN 模型预测与时间无关的商品在未来两周的出货量，部分预测数据如表 8 所示，全部预测结果见（预测结果表 2.6）。

表 8 非季节性商品出货量预测结果

	seller_no	product_no	warehouse_no	date	qty
KNN 算法预测	seller_5	product_2002	wh_1	2023-05-16	2
	seller_5	product_2002	wh_1	2023-05-17	2
	seller_5	product_2002	wh_1	2023-05-18	2
	seller_5	product_2002	wh_1	2023-05-19	2
	...	...	...	...	...

② 更换仓库的商品预测结果

表 9 更换仓库的商品预测结果

	seller_no	product_no	warehouse_no	date	qty
KNN 算法预测	seller_4	product_1683	wh_21	2023-05-16	0
	seller_4	product_1683	wh_21	2023-05-17	1
	seller_4	product_1683	wh_21	2023-05-18	1
	seller_4	product_1683	wh_21	2023-05-19	2
	...	...	...	...	...

上面表 9 中，使用 877 号商品与 1683 号商品作为原始出货量数据，因为在问题一

的商品类别划分中，编号为 877 的商品被判定为与时间无关的商品，则使用与时间无关的商品预测方法—KNN 算法进行预测，部分结果如下表所示。全部预测结果见（预测结果表 2.7）。最终将所有的预测结果整合，得到问题二的总预测结果。（预测结果表 2）

6.3 问题三模型的建立与求解

- **数据处理过程**

Step1 每组商品按时间排序

将所有的商品逐一按照时间排序，得到重新整理好的数据（时间排序表 3.1）。保持仓库、商家、商品一致，查看所给双十一期间各个商品的时间维度是否一致，发现每个商品的数据时间长度在变化并不统一，最短的商品数据只有一条（即：一个日期），分别为编号为 product_892 和 product_920 的两种商品。

Step2 扩充双十一数据

问题三模型建立新表格数据的准备：

（1）将 2023 年 5 月 1 日至 2023 年 5 月 15 日的原始出货量、2023 年 5 月 16 日至 2023 年 5 月 30 日的预测出货量进行数据组合，形成新的数据表（表 3.2）。

（2）提取 2022 年 12 月份的数据，得到模型建立的基本表格数据之一（表 3.3）。

（3）筛选附件 1 中，2022 年 12 月份的数据，将 12 月份作为 10 月份各个商品出货量，并将两个月份的数据进行组合，形成新的数据表（表 3.4）。

（4）汇总表格数据：为了在代码中能同时取到训练集和测试集，并且用于时间序列模型中的 ARIMA 模型数据平稳度检验，以及后期的数据的使用，将 2022 年 11 月份-5 月份商品的出货量数据进行汇总，得到新的数据表（表 3.6）。

问题三中，因为双十一或六月为促销期，居民大都未来折扣，疯狂性的、持续性的购买各类商品，以自用或补贴家用。从十月份到十一月份出货量肯定是增加的，而且从五月份到六月份（618：全民网购狂欢节）也肯定是递增的，所以绝大多数的商品的销售情况肯定与时间有很大的关系。所以，这里用时间序列预测方法中的指数平滑模型。本文在问题三最终选择 stacking 集成算法来综合预测六月份的数据。本次所使用的 stacking 集成算法包括基础模型和元模型两大类，其中，基础模型为：指数平滑模型、ARIMA、KNN、随机森林；元模型为：基于梯度提升的回归模型。

6.3.1 问题三模型的建立

(1) 基础模型的建立

- 指数平滑模型
- ARIMA 模型
- SVM 模型
- 随机森林模型

以上相关模型的介绍以及在问题一中提及，此外问题一提到的**温特斯三重指数平滑模型、移动平均模型预测模型**这里也进行了预测效果的对比，问题一提及到的 KNN 也

做了预测，但是效果不好，因此摒弃了这种方法。同问题一处理一样，我们必须先检验原始训练数据的平稳性，若数据不平稳，必须在应用 AMRIMA 模型前对数据进行差分操作或反差分操作。

对表 3.6 中所有的数据进行平稳性检验，发现原始数据并非平稳序列，因此需要进一步处理，通过差分操作来消除原有的商品出货量数据中隐含的趋势性和季节性因素，从而使得非平稳序列转化为平稳序列。AMRIMA 模型中需要确定三个参数，分别为 p 、q 和 d ，最后一个变量 d 则为差分等级，将其转化为平稳序列。经过调整参数后，我们发现依旧是 d =1（即：一阶差分）的时候效果最好，平稳度达到 90%，并且最大保留了原始数据的信息。如图 20 展示了数据的平稳性非常好。

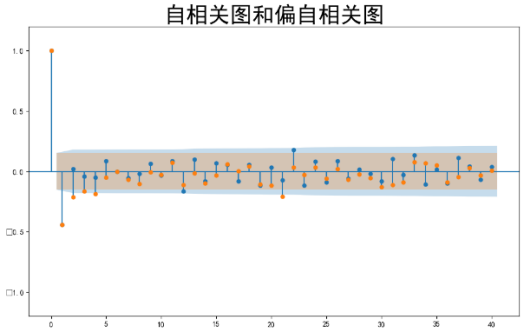
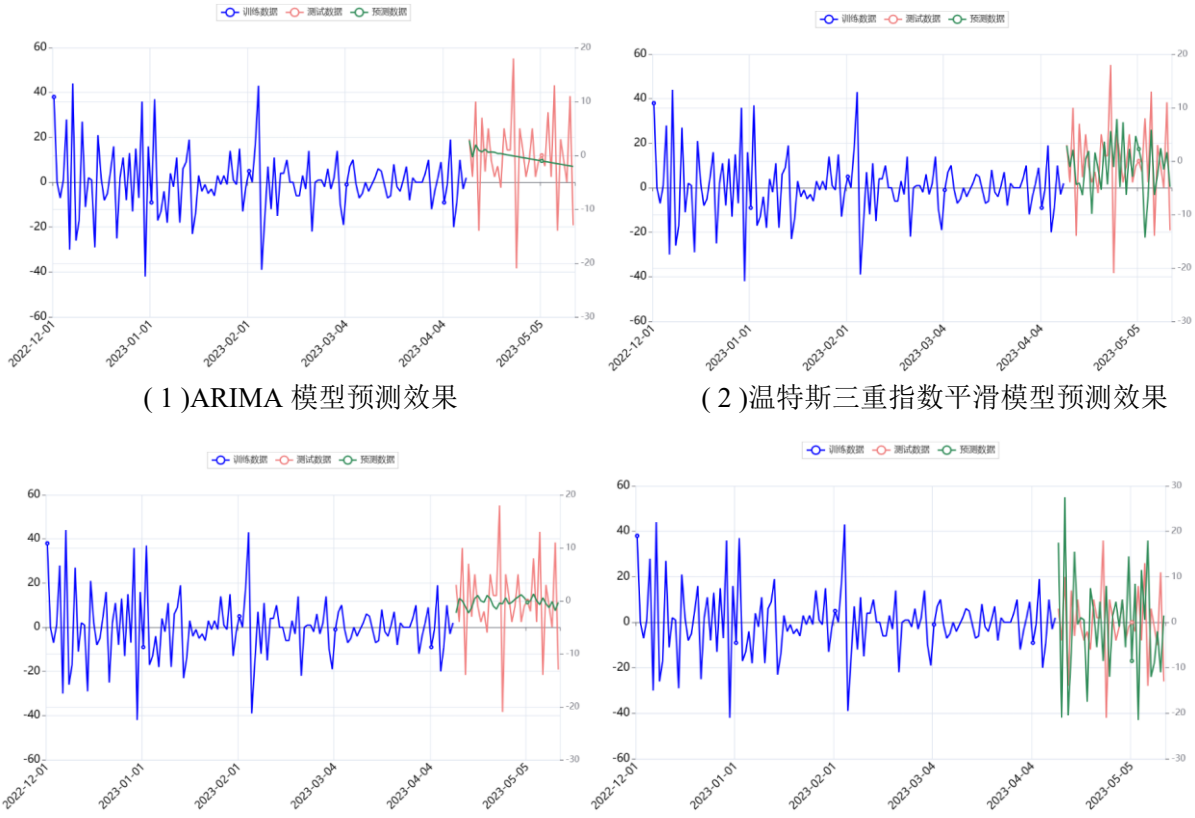


图 20 平稳性效果图

本文以训练集与测试集为 4：1 的比例进行预测，使用基础模型对原始数据进行训练，训练—测试对比图，如图 21 所示。



(1)ARIMA 模型预测效果　　(2)温特斯三重指数平滑模型预测效果

(3)移动平均模型预测效果　　(4)指数平滑预测效果

图 21 基础模型初步预测效果图

(2) 元模型的建立

- 基于梯度提升的回归模型

考虑到单一模型、算法预测总会存在一定的缺陷，都会使得最终六月份的预测数据不准确。因此，可以考虑使用相关的元模型提升算法预测结果的准确度，在一定程度上，对基础模型的预测结果进行相应的整合、处理，以及优化。GBR 原称为：**Gradient Boosting Regressor**，其中 Gradient 即为梯度。它是属于 Boosting 集成算法中的一类用于提升基础模型的回归分析方法。该算法的核心原理是：不断通过迭代，消除预测值和实际值之间的误差值。每进行一次迭代，那么算法就逐步会去拟合目标函数的负梯度方向，也就是残差的方向，最终使得目标函数最小。和 Boosting 算法的本质一样，使用多个弱分类器加起来的作用充当一个强分类器的预测效果。在下面的公式(10)中，展示的为损失函数的通用表达式，它所表示的含义就是使得预测值和实际值之间的差值最小，$y_{(i)}$ 表示实际值，而 $\hat{y}_{(i)}$ 表示估计值（预测值）。所以算法应该沿着 $-\nabla G$ 方向前进。

$$min \quad Lossfunction = \sum_{i=1}^{m} | y_{(i)} - \hat{y}_{(i)} | \tag{10}$$

$$\begin{cases} \nabla G = (\dfrac{\nabla G}{\hat{y}_{(1)}}, \dfrac{\nabla G}{\hat{y}_{(2)}}, \dfrac{\nabla G}{\hat{y}_{(3)}}, ..., \dfrac{\nabla G}{\hat{y}_{(m)}}) \\ \dfrac{\nabla G}{\hat{y}_{(i)}} = \dfrac{\partial (\sum_{i=1}^{m} | y_{(i)} - \hat{y}_{(i)} |)}{\partial \hat{y}_{(i)}} \end{cases} \tag{11}$$

- Stacking 集成学习

Stacking 集成学习是一种分层思想的实现，本文先将原始数据按照 4：1 的比例随机划分为训练集数据和测试集数据。然后将多个训练集数据输入到多个初级的分类器中（基础模型），然后将测试集输入到已经训练好的初级模型中去，将所有预测的输出结果值进行加权（线性组合），又作为输入值对第二级模型（元模型）进行训练，最终得到输出结果。Stacking 是在相同数据、不同模型下训练不一样的模型，具体流程如图 22。

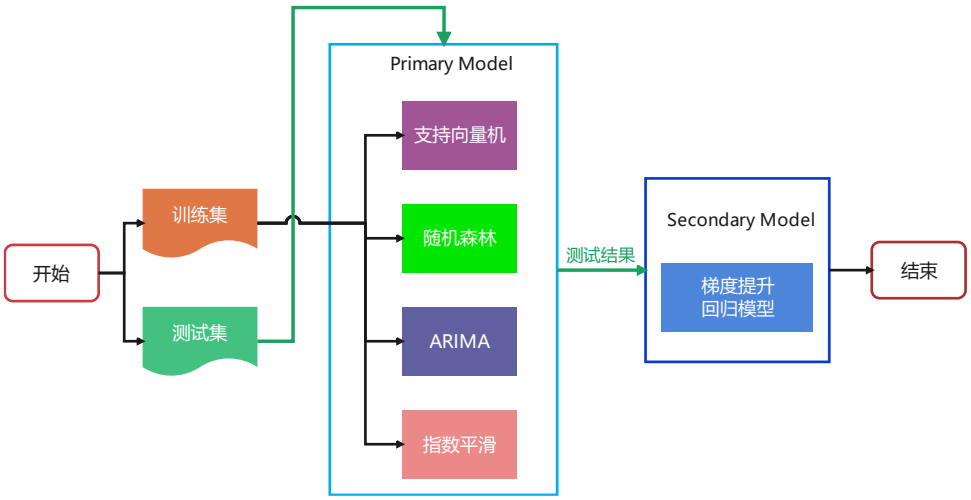


图 22　Stacking 融合模型思路图

6.3.2 问题三模型的求解

分别使用基于基础模型（SVM、ARIMA、随机森林、指数平滑）和元模型（GBR）进行 Stacking 融合多模型的思路，预测出未来 2023 年 6 月 1 日至 6 月 20 号的各个商品出货量。部分预测结果见下表所示，全部预测结果见（预测结果表 3）。

表 10 六月份商品出货量预测结果

seller_no	product_no	warehouse_no	date	forecast_qty
seller_10	product_1914	wh_1	2023-06-01	2
seller_10	product_1914	wh_1	2023-06-02	1
seller_10	product_1914	wh_1	2023-06-03	2
seller_10	product_1914	wh_1	2023-06-04	2
seller_10	product_1914	wh_1	2023-06-05	1
seller_10	product_1914	wh_1	2023-06-06	2
seller_10	product_1914	wh_1	2023-06-07	2
...	...	...	...	...

6.3.3 问题三模型的评估

对于各个商品种类，我们均建立了随机森林、SVM、ARIMA、指数平滑模型，并且进行迭代得到最优参数，得到每一个方法的最优模型，对此我们用以下指标来评估准确率。

- 平均准确率

$$1-p=1-\frac{1}{n}\cdot\frac{\sum_{i=1}^{n}|y(i)-\hat{y(i)}|}{\sum_{i=1}^{n}y(i)} \tag{12}$$

计算式(12)中，当$1-p$的结果越接近于 1 时，代表预测效果更好。

- 准确率

$$\text{Precision}=\frac{TP}{TP+FP} \tag{13}$$

计算式(13)中，Precision 代表预测正确样本的概率，FP代表预测的样本总数，TP代表预测正确的样本数。当 Precision 的结果越接近于 1 时，代表预测效果更好。

- 均方误差

$$MSE=\frac{1}{n}\cdot\sum_{i=1}^{n}(y(i)-\hat{y(i)})^2 \tag{14}$$

计算式(14)中，当MSE的结果越接近于 0 时，代表预测效果更好。

通过上述所建立的三个评价指标，计算出每个模型下，某几种商品出货量的三个指标大小，最终结果如表 11、表 12、表 13 所示。

为了使用各项评估指标对模型的好坏进行检验，我们使用相应的商品原始的出货量数据和现有的预测出货量数据进行各项评分计算，对此，我们随机抽取商品进行指标评分的计算。表 11 为各个商品在所建立的随机森林、SVM 模型三项指标评分结果；表 11 为各个商品在所建立的指数平滑、ARIMA 模型三项指标评分结果；表 12 为对以上基础

模型增加元模型后的 stacking 集成学习各类评价指标评分。

表 11 随机森林、SVM 三项指标评分结果

待评价商品编号	随机森林			SVM		
	平均准确率	准确率	均方误差	平均准确率	准确率	均方误差
product_1914	0.588	0.692	2.355	0.724	0.736	2.028
product_1915	0.633	0.376	1.573	0.628	0.637	2.993
product_1916	0.573	0.548	2.634	0.642	0.545	1.993
...	...	...	...	...	...	...

表 12 指数平滑、ARIMA 三项指标评分结果

待评价商品编号	指数平滑			ARIMA		
	平均准确率	准确率	均方误差	平均准确率	准确率	均方误差
product_1914	0.738	0.842	3.355	0.514	0.536	2.258
product_1915	0.543	0.636	1.743	0.728	0.647	2.233
product_1916	0.743	0.548	2.034	0.542	0.735	1.793
...	...	...	...	...	...	...

表 13 stacking 集成学习后效果表

基础模型	使用 GBR 模型进行 stacking 集成学习后整体平均提升值或下降值					
	平均准确率	准确率	均方误差（下降）	平均准确率	准确率	均方误差
随机森林	0.119	0.154	0.992	0.213	0.243	0.243
SVM	0.232	0.523	1.011	0.232	0.262	0.453
指数平滑	0.210	0.231	0.142	0.192	0.143	0.569
ARIMA	0.277	0.344	0.466	0.228	0.119	0.253

通过表 11、表 12 到表 13 各项模型对应的三个指标的数值跃变，体现出使用 GBR 模型进行 stacking 集成学习后整体模型预测更准确，且 MSE（均方误差）越小，预测出的六月份促销期间的出货量准确度高，使用 stacking 集成学习模型效果提升显著。

6.4 库存优化问题的设想

商品的库存应该不多也不少。库存过多，会增加商品储存的成本，即持有成本；而库存过少，则可能会由于仓库租金续约、仓库员工工资结算等现实问题导致缺货成本的增加。因此必须尽可能的减少损失。这里我们可以猜想采用线性规划或多目标规划等模型来解决这一优化类问题。

为此，设总持有成本为hold_cost，总缺货成本为Lack_cost，补货量为Q已知商品的盘点库存为T，根据题意，T 总共存在以下两种情况：

$$\begin{cases} 0 \le T < s & Q = S - T \\ s \le T < S & Q = 0 \end{cases} \tag{15}$$

设客户需求量为$client_quanity$，而仓库的现存商品量为T，若$T < client_quanity$，则商品会出现短缺损失，计算式(16)中t_1代表缺货时间，LT表示最大缺货时间（补货运输时间）则总的缺货成本为：

$$\begin{cases} \text{Lack_cost} = (client_quanity - T) \times h \times t_1 \\ t_1 \le LT \end{cases} \tag{16}$$

每件商品自入库以来，从第一天算起，就带有一定的持有成本，因此一般商家要尽快的向广大客户销售出去的商品，防止积压。设原先进货量为S，其中计算式(17)t_2表示原先已经销售出去的商品在仓库中滞留的时间，t_3表示现有的库存量需要直到卖出去的天数。则总的持有成本为：

$$\text{hold_cost} = (S - T) \times h \times t_2 + T \times h \times t_3 \tag{17}$$

另外，库存周转天数的含义是：该商家从进货或者商品入库开始，直到该商品全部销售为止的天数。库存周转天数是衡量该企业运营能力的一个重要指标；库存周转天数越小，代表该仓库的进货、出货量的频次越高，商品的供应链极其稳定，体现了该企业存货的变现能力之高。因此，该问题相当于基于上述限制条件下，使得目标函数达到最小值。

$$min \quad \begin{cases} \text{Lack_cost} \\ \text{hold_cost} \end{cases} \tag{18}$$

七、模型的评价与推广

7.1 模型的评价

- 模型的优点

1. 数据处理的严谨性

（1）对数据异常值进行剔除或更换能确保预测结果更准确。

（2）考虑到数据间的量纲不一致问题，将文本型数据转化为数值型数据，使得处理更加方便。

（3）对数据进行标准化、归一化处理，不仅没有改变数据的原始信息，同时使得各个特征在同一水平下比较。例如：在本文中，对权值进行归一化，使得权值按照一定的比例配比，得到各个特征的相对重要性。

2. 应用模型的严谨性

（1）对于应用 ARIMA 模型预测时，应模型本身的限制要求，先对数据进行了平稳性检验，例如：本文在处理问题三的时候检测出原始数据呈现非平稳性，于是使用一阶差分法对其进行了处理。

（2）对于同一种预测或分类问题，本文使用多种方法进行对比，最终选取最优的方法进行预测。例如：在问题一的分类问题中，使用 Kmeans 聚类结果只能分

为两类；而 DBSCAN 可以分为多类，并且可视化效果更好。

（3）在研究分类问题时候，使用多个特征列来构建特征工程时，先使用主成分分析对数据进行降维处理，提取关键信息。既加快了算法的收敛速度，也使得问题变得更容易处理，并且考虑到所研究问题是时间序列，则主观加大时间的权重，使得结果更为合理。

- 模型的缺点

（1）对于的预测方法的选择存在一定的主观性，不同的预测方法预测所得结果会存在差异。

（2）对于赋予的权重，各个特征向量所占的比例不同，对后期预测结果有一定的影响。

- 模型的改进

（1）在问题三中，本文默认促销期间所有的商品都与时间有关，所以并没有使用一定的分析来特殊处理离群值，很可能一些商品随着时间呈现不规则变化，与时间并无关系。

（2）可以适当的调整所使用的基础模型的种类或个数，使得初级分类器预测的效果更好。

（3）对相应的特征向量可以加入更多的限制条件或者参数，使得预测结果更合理。在合理的情况下，从外界获取更多的数据，用于合理性预测。

7.2 模型的推广

DBSCAN 是区别于聚类算法、基于数据点密度划分数据类别的一种算法，作为一种比较新型的分类算法，DBSCAN 算法在计算机科学、人工智能、数据挖掘、图像识别、图像处理、语义识别等研究领域应用广泛。它对异常值的存在并不受很大的影响，并且算法的运算速度也较快，所以在新生物科技领域，畅想它对于病毒的快速分类也能起到一定的辅助作用。当有病毒出现时，科学家可以拿出相应的武器，将 DBSCAN 算法与机器算法结合，利用 DBSCAN 模型提取病毒的特征，然后使用适合的高级机器学习算法对病毒进行分类。另外；对于人类或其他生物的基因方面的研究与探索至今没有停止过，未来甚至猜想可以将 DBSCAN 算法与生物信息库、信息技术相结合，准确的进行基因序列的规律性与原理性分析，更好的识别或者分类病毒。

参考文献

[1] 申伟.基于数据驱动的海上风电场智能监测与运维研究[D].齐鲁工业大学,2023.

[2] 饶雷,冉军,陶建权等.基于随机森林的海上风电机组发电机轴承异常状态监测方法[J].船舶工程,2022,44(S2):27-31.

[3] 赖宝,万仁霞,赵杰等.基于三支决策的支持向量机分类算法[J].闽南师范大学学报(自然科学版),2023,36(03):22-30.

[4] 周晓晔,成佳慧,戴思聪.基于 Holt-Winters 和 PSO-SVR 的快递业务量最优加权组合预测[J].数学的实践与认识,2023,53(07):51-60.

[5] 杜青松,李慧,刘振渤等.基于 DBSCAN 算法的出租车载客热点分析[J].现代信息科技,2023,7(19):89-93+98.

附录

注明：

1. 所有源程序基于 Jupyter 编译器编写，每段源程序的运行基于一定的路径设置。
2. 由于考虑到篇幅、及美观性，这里粘贴每一个问题的核心源程序，其余所有的源程序见支撑材料

附录 1

介绍：数据预处理

```
'''
数据预处理
'''
import pandas as pd
# 读取 Excel 文件
file_path = r"C:\Users\BaiLu\Desktop\新建文件夹 (2)\附件 6-促销期间商家出货量表.xlsx"# 替换成你的 Excel 文件路径
df = pd.read_excel(file_path)
# 指定哪些列用于判断重复
columns_to_check = ['seller_no', 'product_no', 'warehouse_no','date']
# 找出重复行
duplicates = df[df.duplicated(subset=columns_to_check, keep=False)]
# 打印重复行
print(duplicates)
'''
对和时间没有关系的数据进行预测
'''
import pandas as pd
import seaborn as sns
'''
识别异常值
'''
new_df6= result_df.groupby(['product_no'])
for product_no, group_data in new_df6:
    # 识别异常值
    Q1 = group_data['qty'].quantile(0.25)
    Q3 = group_data['qty'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = group_data[(group_data['qty'] < lower_bound) |
(group_data['qty'] > upper_bound)]
    # 4. 替代异常值为平均值
    mean_value = group_data['qty'].mean()
    result_df.loc[outliers.index, 'qty'] = mean_value
```

附录 2

介绍：问题一的处理（每段源程序运行基于一定的路径设置）

```
'''
商品按时间分组排序
'''
# 读取文件
file_path = r"C:\Users\BaiLu\Desktop\2023 年 MathorCup 大数据挑战赛-初赛赛题\2023 年 MathorCup 大数据挑战赛-赛道 B 初赛\附件数据及提交结果表格\附件表\附件 1-商家历史出货量表.xlsx"  # 替换为你的文件路径
df = pd.read_excel(file_path)
'''
相关系数分析，以 0.6 为阈值提取出和时间有关系的那些商品
'''
from scipy import stats
# 计算 Pearson 相关系数
selected_groups =[]
for i in range(1996):
    df_0 = result.iloc[1+i * 166:1+(i+1)*166,:]
    # 将日期列转换为日期对象
    df_0['date'] = pd.to_datetime(df_0['date'])
    # 将日期转换为整数表示
    df_0['date'] = (df_0['date'] - df_0['date'].min()).dt.days
    correlation, p_value = stats.pearsonr(df_0['date'], df_0['qty'])
    # 如果相关性大于阈值，将组别添加到 selected_groups 中
    if abs(correlation) > 0.6:
        selected_groups.append({'seller_no':df_0['seller_no'].values[0],
'product_no':df_0['product_no'].values[0],'warehouse_no':
df_0['warehouse_no'].values[0]})
groups = pd.DataFrame(selected_groups)
print("满足相关性阈值的组别：")
print(groups)
path  =  (r"C:\Users\BaiLu\Desktop\2023 年 MathorCup 大数据挑战赛-初赛赛题\2023 年 MathorCup 大数据挑战赛-赛道 B 初赛\附件数据及提交结果表格\附件表\相关系数表.xlsx")
with pd.ExcelWriter(path, engine='openpyxl') as writer:
    groups.to_excel(writer, sheet_name='Sheet1', index=False)
'''
ARIMA 模型：非平稳性检验
'''
import pandas as pd
from statsmodels.tsa.stattools import adfuller
# 读取时间序列数据
# 假设您的时间序列数据存储在一个名为 "time_series_data.csv" 的 CSV 文件中，其中有一列名为 "value" 的数据。
df_ = merged_data.iloc[0:166,:]
time_series = df_ ["qty"]
```

```
# 进行ADF检验
result1 = adfuller(time_series)
# 输出ADF检验结果
adf_statistic, p_value, _, _, _, _ = result1
print(f"ADF统计量：{adf_statistic}")
print(f"P值：{p_value}")
# 根据P值进行判断
if p_value <= 0.05:
    print("时间序列数据是平稳的，无需差分。")
else:
    print("时间序列数据不是平稳的，需要进行差分。")
'''
使用时间序列预测中的ARIMA类的模型，改变不同的参数以达到最佳的拟合效果

'''
import matplotlib.dates as mdates
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
# 划分训练集和测试集
train_size = int(len(time_series_data1) * 0.8)
train_data1,test_data1=time_series_data1[:train_size],
time_series_data1[train_size:]
train_size = int(len(time_series_data1) * 0.8)
train_data,test_data=time_series_data[:train_size],
time_series_data[train_size:]
best_model = None
best_mse = float('inf')
# 尝试不同的模型
for p in range(1, 4):
    for d in range(1, 3):
        for q in range(1, 4):
            # 使用ARIMA模型进行拟合
            model = ARIMA(train_data1, order=(p, d, q))
            model_fit = model.fit()
            # 测试集进行效果检验
            forecast_steps = len(test_data1)
            forecast = model_fit.forecast(steps=forecast_steps)
            # 计算均方根误差
            mse = mean_squared_error(test_data1, forecast)
            if mse < best_mse:
                best_model = model_fit
                best_mse = mse
# 使用最佳模型进行预测
forecast = best_model.forecast(steps=forecast_steps)
"""
```

```
寻找最大的指数平滑的参数，使得模型效果最好
"""
import matplotlib.dates as mdates
from statsmodels.tsa.holtwinters import ExponentialSmoothing
res00 = dict()
for i in range(2,66):
    model      =      ExponentialSmoothing(train_data,      seasonal='add',
seasonal_periods=i)  # 这里使用季节性指数平滑
    model_fit = model.fit()
    # 测试集进行效果检验
    forecast = model_fit.forecast(steps=forecast_steps)
    # 计算均方根误差
    mse = mean_squared_error(test_data, forecast)
    res0=1-sum(np.array(abs(test_data.values
forecast.values)))/sum(np.array(test_data.values))
    res00[str(i)] = res0
    print(f"{i}:{res0}")
    i = 1 + i
# 初始化最小正数和对应的键
min_positive_value = None
min_positive_key = None
# 遍历字典
for key, value in res00.items():
    if value > 0:  # 检查是否为正数
        if min_positive_value is None or value < min_positive_value:
            min_positive_value = value
            min_key = key
if min_positive_key is not None:
    print(f"正数中最小的数对应的键是: {min_key}")
"""
指数平滑预测
"""
from pyecharts.charts import Line
from pyecharts import options as opts
from pyecharts.commons.utils import JsCode
# 示例参数和对应的MSE值，需要替换为你的实际数据
parameters = list(res00)  # 参数值
mse_values = list(res00.values())  # 对应的MSE值
mse_values = [abs(x) for x in mse_values]
# 寻找最小MSE值和对应的参数
min_mse = min(mse_values)
min_mse_index = mse_values.index(min_mse)
min_mse_parameter = parameters[min_mse_index]
import matplotlib.dates as mdates
from statsmodels.tsa.holtwinters import ExponentialSmoothing
# 使用指数平滑模型进行拟合
```

```
model = ExponentialSmoothing(train_data, seasonal='add', seasonal_periods=5)  # 这里使用季节性指数平滑
model_fit = model.fit()
# 测试集进行效果检验
forecast = model_fit.forecast(steps=34)
# 计算均方根误差
mse = mean_squared_error(test_data, forecast)
print(f'Mean Squared Error (MSE): {mse}')

import matplotlib.dates as mdates
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
import numpy as np
"""
温特斯三重指数平滑预测
"""
model=ExponentialSmoothing(train_data, seasonal='add', seasonal_periods=30, trend='add')  # 使用季节性和趋势指数平滑
model_fit = model.fit()
# 测试集进行效果检验
forecast = model_fit.forecast(steps=forecast_steps)
'''
使用移动平均进行拟合预测
'''
import matplotlib.dates as mdates
import pandas as pd
import matplotlib.pyplot as plt
# 计算移动平均
window_size = 7  # 设置移动平均窗口大小
rolling_mean = train_data.rolling(window=window_size).mean()
'''
对和时间没有关系的数据进行预测
'''
file_path = r"C:\Users\BaiLu\Desktop\2023 年 MathorCup 大数据挑战赛-初赛赛题\2023 年 MathorCup 大数据挑战赛-赛道 B 初赛\附件数据及提交结果表格\附件表\1.xlsx"
# 替换为你的文件路径
df0 = pd.read_excel(file_path)
file_path1 = r"C:\Users\BaiLu\Desktop\2023 年 MathorCup 大数据挑战赛-初赛赛题\2023 年 MathorCup 大数据挑战赛-赛道 B 初赛\附件数据及提交结果表格\附件表\相关系数表 2.xlsx"  # 替换为你的文件路径
df1 = pd.read_excel(file_path1)
'''
Bagging 随机森林预测
'''
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestRegressor
# 从数据框中分别提取名为“date”和“qty”的列
date_column = result_df['date']
qty_column = result_df.iloc[:166,:]['qty']
# 将提取的列转换为NumPy 数组
X = np.array(list(range(1, 167)))
X=X.reshape(-1,1)
y = np.array(qty_column)
y=y.reshape(-1,1)
# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# 创建随机森林实例
clf = RandomForestRegressor(n_estimators=100, random_state=42)
# 使用训练集训练模型
clf.fit(X_train, y_train)
# 使用测试集进行预测
y_pred = clf.predict(X_test)
print(y_pred)
residuals = np.array(abs(y_test - y_pred ))
accuracy = 1 - sum(residuals) / sum(np.array(y))

print(f'拟合精度: {np.mean(accuracy)}')
'''
KNN 回归预测
'''
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score
# 从数据框中分别提取名为“date”和“qty”的列
date_column = result_df['date']
qty_column = result_df.iloc[:166, :]['qt
# 将提取的列转换为NumPy 数组
X = np.array(list(range(1, 167)))
X = X.reshape(-1, 1)
y = np.array(qty_column)
#y = y.reshape(-1, 1)
# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# 创建KNN 回归模型实例
knn = KNeighborsRegressor(n_neighbors=2)  # 可以调整邻居数
# 使用训练集训练模型
knn.fit(X_train, y_train)
# 使用测试集进行预测
y_pred = knn.predict(X_test)
```

```
print(y_pred)
'''
支持向量机预测
'''
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.metrics import r2_score
# 从数据框中分别提取名为“date”和“qty”的列
date_column = result_df['date']
qty_column = result_df.iloc[:166, :]['qty']
# 将提取的列转换为NumPy 数组
X = np.array(list(range(1, 167)))
X = X.reshape(-1, 1)
y = np.array(qty_column)
# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# 创建SVM 回归模型实例
svm = SVR(kernel='linear')  # 可以选择不同的核函数
# 使用训练集训练模型
svm.fit(X_train, y_train)
# 使用测试集进行预测
y_pred = svm.predict(X_test)
print(y_pred)
```

附录 3

介绍：问题二的处理（每段源程序运行基于一定的路径设置）

```
'''
提取改变仓库了的商品
'''
merged = pd.merge(df2, df0, on=['seller_no', 'product_no'], how='left',
indicator=True)
result = merged[merged['_merge'] == 'left_only']
# 删除掉"_merge"列，它只是一个指示符
result = result.drop(columns=['_merge'])
result=result.drop(columns=["warehouse_no_y","date_y","qty_y"])
# 创建一个字典来定义新的列名
new_columns = {'warehouse_no_x': 'warehouse_no', 'product_no':
'product_no',"date_x":"date","qty_x":"qty"}
# 使用rename 方法更改列名
result= result.rename(columns=new_columns)
new_df = result
'''
提取新增的商品
'''
```

```
# 使用 merge 函数以 key 列为键合并两个数据框，使用 outer 模式
merged_data = pd.merge(df2,result,  on=['seller_no', 'product_no',
'warehouse_no', 'date', 'qty'], how='outer', indicator=True)
different_rows = merged_data[merged_data['_merge'] == 'left_only']
result_df = different_rows.drop(columns=['_merge'])
change_df = result_df
'''
与时间有关的新增商品
'''
from scipy import stats
# 先进行分组
new_df1= new_df.groupby(['product_no'])
selected_groups =[]
# 迭代分组并查看每个分组的数据
for product_no, group_data in new_df1:

    # 将日期列转换为日期对象
    group_data['date'] = pd.to_datetime(group_data['date'])
    # 将日期转换为整数表示
    group_data['date'] = (group_data['date'] -
group_data['date'].min()).dt.days
    correlation, p_value = stats.pearsonr(group_data['date'],
group_data['qty'])
    # 如果相关性大于阈值，将组别添加到selected_groups 中
    if abs(correlation) > 0.6:
        selected_groups.append({'seller_no':
group_data['seller_no'].values[0], 'product_no': product_no,
'warehouse_no': group_data['warehouse_no'].values[0]})
groups = pd.DataFrame(selected_groups)
print("满足相关性阈值的组别：")
print(groups)
# 使用merge 方法合并数据框，根据特定列进行匹配
merged_data = df2.merge(groups, on=['seller_no', 'product_no',
'warehouse_no'])
# 现在你可以在merged_data 中访问匹配条件的行
path = (r"C:\Users\BaiLu\Desktop\2023 年 MathorCup 大数据挑战赛-初赛赛题\2023
年 MathorCup 大数据挑战赛-赛道 B 初赛\附件数据及提交结果表格\附件表\与时间有关的商
品 2.xlsx")
with pd.ExcelWriter(path, engine='openpyxl') as writer:
    merged_data.to_excel(writer, sheet_name='Sheet1', index=False)
'''
对和时间有关系的数据进行预测
'''
import matplotlib.dates as mdates
from statsmodels.tsa.holtwinters import ExponentialSmoothing
new_df2= merged_data.groupby(['product_no'])
```

```
merged_df = pd.DataFrame()
for product_no, group_data in new_df2:
    date_rng = pd.to_datetime(group_data['date'])
    data = np.array(group_data['qty'])
    time_series_data = pd.Series(data, index=date_rng)
    # 使用指数平滑模型进行拟合
    model = ExponentialSmoothing( time_series_data, seasonal='add',
seasonal_periods=5)  # 这里使用季节性指数平滑
    model_fit = model.fit()
    # 测试集进行效果检验
    # 指定日期
    specified_date = pd.to_datetime('2023-05-30')
    # 计算日期差
    steps_num = (specified_date - date_rng.max()).days
    # 预测
    forecast = model_fit.forecast(steps=steps_num)
'''
用时间序列相似性度量方法，如动态时间规整（Dynamic Time Warping，DTW
'''
dis = pd.DataFrame()
for product_no, group_data in grouped:
    df_22 = pd.DataFrame()
    date1 = group_data['date'].iloc[:127]
    date2 = group_data['date'].iloc[127:]
    qty1 = group_data['qty'].iloc[127:]
    qty_duibi = df2['qty']
#     # 转换日期数据为时间戳并计算距离
#     # 将 qty1 和 qty_duibi 转换为一维数组
#     qty1 = qty1.values.ravel()
#     qty_duibi = qty_duibi.values.ravel()
    date1_timestamp = qty1.apply(lambda x:
pd.to_datetime(x).timestamp()).values.reshape(-1, 1)
    date2_timestamp = qty_duibi.apply(lambda x:
pd.to_datetime(x).timestamp()).values.reshape(-1, 1)
    # 使用 DTW 算法计算 se1 与 se2 的距离
    distance1, _ = fastdtw(date1_timestamp ,  date2_timestamp,
dist=euclidean)
    df_22 = pd.DataFrame({
        "seller_no": np.array(group_data["seller_no"]),
        "product_no": np.array(group_data["product_no"]),
        "warehouse_no": np.array(group_data["warehouse_no"]),
        "dis":np.array(distance1)
    })
    # 将 df_22 合并到 merged_df
    dis  = pd.concat([ dis, df_22],axis = 0)
# 创建 KNN 回归模型实例
```

```
knn = KNeighborsRegressor(n_neighbors=2)  # 可以调整邻居数

# 使用训练集训练模型
knn.fit(X, y)
#预测集
x = np.arange(166,166+15)
x = x.reshape(-1, 1)
# 使用测试集进行预测
y_pred = knn.predict(x)
```

附录 3

介绍：问题三的处理（每段源程序运行基于一定的路径设置）

```
'''
将 12 月的表和需要预测的十一月份的表合起来，并将 12 月替换为 10 月
'''
group = df.groupby(['seller_no', 'warehouse_no', 'product_no'])
fina = pd.DataFrame()
for product_no, group_data in group :
    combined_series = pd.Series()
    seller_no = group_data["seller_no"].iloc[0]
    product_no = group_data["product_no"].iloc[0]
    warehouse_no= group_data["warehouse_no"].iloc[0]
    selec1 = month10_11[(month10_11["seller_no"] == seller_no) & (month10_11["product_no"] == product_no) & (month10_11["warehouse_no"] == warehouse_no)]
    # 使用 concat 函数进行垂直合并
    group_data['date'] = group_data['date'].astype(str).str.split().str[0]
    combined_series = pd.concat([selec1, group_data], axis=0)
    fina = pd.concat([fina, combined_series])
'''
将处理好的数据进行预测，month10_11 是用来训练的，month5 是用来预测的前半段，相当于 month10
'''
import pandas as pd
import numpy as np
# 先进行分组
grouped = fina.groupby(['seller_no', 'warehouse_no', 'product_no'])
fina1 = pd.DataFrame()
# 指数平滑
def exponential_smoothing(data, alpha,days):
    model = ExponentialSmoothing(data, trend='add', seasonal='add', seasonal_periods=3,freq='D')
    model_fit = model.fit(smoothing_level=alpha, smoothing_slope=alpha, smoothing_seasonal=alpha)
    forecast = model_fit.forecast(steps=days)
    return forecast.values
```

```
# ARIMA
def arima(data, p, d, q,days):
    model = ARIMA(data, order=(p, d, q))
    model_fit = model.fit()
    forecast = model_fit.forecast(steps=days)
    return forecast
# SVM
def svm(data,days):
    model = SVR()
    # 获取日期索引并将其转换为数值型特征
    array = np.arange(len(data)).reshape(len(data), 1)
    X = array.reshape(-1, 1)
    y = data
    # 模型训练
    model.fit(X, y)
    X_test= np.arange(len(data),len(data) + days).reshape(-1,1)
    # 进行预测
    forecast = model.predict(X_test)
    return forecast
# 随机森林
def random_forest(data,days):
    model = RandomForestRegressor(n_estimators=100, random_state=0)
    # 使用日期的数值表示来作为特征
    X = np.arange(len(data)).reshape(-1,1)
    y = data
    model.fit(X, y)
    # 生成预测
    test_X = np.arange(len(data),len(data) + days).reshape(-1,1)
    forecast = model.predict(test_X)
    return forecast
def reshape_forecast(forecast):
    return forecast.reshape(-1, 1)
# 使用每个基础模型生成预测结果
ma_window_size = 7
alpha = 0.05
p, d, q = 1, 1, 1
merged_df= pd.DataFrame()
#使用stacking算法进行预测
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.svm import SVR
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.ensemble import GradientBoostingRegressor
for product_no, group_data in grouped :
    df_22 = pd.DataFrame()
    # 对data进行处理
    date_rng = group_data['date']
    data0 = np.array(group_data['qty'])
    time_series_data = pd.Series(data0, index=date_rng)
    # 假设日期列的名称为"date"，确保它的数据类型是日期时间类型
    group_data['date'] = pd.to_datetime(group_data['date'])
    # 使用条件筛选找出2022年11月份的数据
    november_2022_data = group_data[(group_data['date'].dt.year == 2022) &
(group_data['date'].dt.month == 11)]
    # 找出2022年11月份数据中的最大日期
    max_date_november_2022 = november_2022_data['date'].max()
    # 划分数据集
    train_data = time_series_data['2022-11-01':str(max_date_november_2022)]
    start_date = '2023-06-01'
    end_date = '2023-06-20'
    date_range = pd.date_range(start=start_date, end=end_date)
    test_data  =date_range
    days = len(test_data)
    Y_train = train_data.values
    data = Y_train
    X_train = np.arange(len(train_data)).reshape(-1,1)
    start_date = '2023-06-01'
    end_date = '2023-06-20'
    date_range = pd.date_range(start=start_date, end=end_date)
    test_data=  np.arange(len(data),len(data)  +  len(date_range)).reshape(-
1,1)
    # 预测
    if len(Y_train)>= 6:
        exp_smooth_forecast = exponential_smoothing(train_data, alpha,days)
        arima_forecast = arima(Y_train, p, d, q,days)
    svm_forecast = svm(Y_train,days)
    random_forest_forecast = random_forest(Y_train,days)
    # 使用基础模型生成预测结果
    if len(Y_train)>= 6:
        exp_smooth_forecast_test     =     exponential_smoothing(train_data,
alpha,days)
        arima_forecast_test = arima(Y_train, p, d, q,days)
    svm_forecast_test = svm(Y_train,days)
    random_forest_forecast_test = random_forest(Y_train,days)
    # 在调用各种预测函数之前，对它们的输出进行形状调整
    if len(Y_train)>= 6:
```

```
        exp_smooth_forecast = reshape_forecast(exp_smooth_forecast)
        arima_forecast = reshape_forecast(arima_forecast)
    svm_forecast = reshape_forecast(svm_forecast)
    random_forest_forecast = reshape_forecast(random_forest_forecast)
    # 创建元特征
    if len(Y_train)>= 6:
        meta_features     =     np.column_stack((     exp_smooth_forecast,
arima_forecast, svm_forecast, random_forest_forecast))
    else:
        meta_features     =     np.column_stack((           svm_forecast,
random_forest_forecast))
    # 定义元模型
    # 创建 GradientBoostingRegressor 对象
    meta_model        =        GradientBoostingRegressor(n_estimators=100,
learning_rate=0.1, max_depth=3)
    # 训练元模型
    meta_model.fit(X_train ,Y_train)
    # 使用元模型生成最终预测
    if len(Y_train)>= 6:
        #meta_features_test  =  np.column_stack((  exp_smooth_forecast_test,
arima_forecast_test, svm_forecast_test, random_forest_forecast_test))
        final_predictions                                                =
meta_model.predict(exp_smooth_forecast_test.reshape(-1, 1))
    else:
        #meta_features_test    =    np.column_stack((    svm_forecast_test,
random_forest_forecast_test))
        final_predictions  =  meta_model.predict(svm_forecast_test.reshape(-
1, 1))
    min_length = 20
    df_22 = pd.DataFrame({
        "seller_no":
np.array(group_data["seller_no"].iloc[:min_length]).flatten(),
        "product_no":
np.array(group_data["product_no"].iloc[:min_length]).flatten(),
        "warehouse_no":
np.array(group_data["warehouse_no"].iloc[:min_length]).flatten(),
        "date": np.array(date_range).flatten(),
        "qty": np.array(final_predictions).flatten()
    })
    # 将 df_22 合并到 merged_df
    merged_df = pd.concat([merged_df, df_22],axis = 0)
# 输出合并后的 DataFrame
print(merged_df)
```